

BarakVPN



תלמיד: ברק רוזנפלד

בית ספר: תיכון אלון

ת"ז: 331639203

מנחה: עידן פלדברג

חלופה: תוכנות VPN

תאריך הגשה: 24.5.25



רמת השרון תיכון אלון לאמנויות ולמדעים



תוכן עניינים

1.....	תוכן עניינים
4.....	מבוא
4.....	ייזום:
4.....	תיאור ראשוני של המערכת
4.....	הגדרת הלקוח
4.....	הגדרת יעדים
5.....	בעיות, תועלות וחסכונות
5.....	סקירת פתרונות קיימים
5.....	סקירת טכנולוגית הפרויקט
6.....	תיחום הפרויקט
6.....	פירוט תיאור המערכת (אפיון):
6.....	תיאור מפורט יותר של המערכת
6.....	פירוט היכולות שהיא תעניק לכל סוג משתמש במערכת
7.....	טבלת הבדיקות המתוכננות
8.....	תכנון וניהול לו"ז זמנים לפיתוח המערכת:
9.....	ניהול הסיכונים בפרויקט והדרכים להתמודדות איתם
9.....	תיאור תחום הידע - פרק מילולי
9.....	יכולות המשתמש
9.....	יכולות בצד שרת
10.....	פירוט:
10.....	• התחברות למערכת -
11.....	• הרשמה מאובטחת למערכת
11.....	• ביצוע חיפושים רגילים
12.....	• ביצוע חיפוש לפי קישור -
13.....	• פתיחת קישור לצפייה בדפדפן מקומי
14.....	• שמירת תוכן הכתבות בקובץ טקסט
15.....	מבנה / ארכיטקטורה של הפרויקט
15.....	רכיבי המערכת:
16.....	הטכנולוגיה הרלוונטית:
17.....	זרימת המידע במערכת
17.....	1. רישום משתמש (Sign Up):
17.....	2. התחברות משתמש (Login):
17.....	3. חיפוש כתבות:
18.....	4. צפייה בכתבה (Open in browser):
18.....	תיאור האלגוריתמים מרכזיים בפרויקט:
18.....	בעיה אלגוריתמית 1: השגת קישורים מהרשת
19.....	בעיה אלגוריתמית 2: סינון כתבות שאינן ניתנות לקריאה
19.....	אתגרים:
19.....	פתרונות קיימים:

20.....	הפתרון שנבחר בפרויקט:
20.....	נימוק הבחירה:
20.....	בעיה אלגוריתמית 3: שליחת תוכן ארוך מהשרת ללקוח
20.....	אתגרים:
21.....	תיאור סביבת הפיתוח
21.....	כלי הפיתוח הדרושים לפיתוח המערכת:
21.....	סביבה וכלים לבדיקות:
22.....	תיאור פרוטוקול התקשורת
22.....	תיאור מילולי של פרוטוקול התקשורת:
22.....	סוגי ההודעות הזורמות במערכת:
23.....	תיאור מסכי המערכת:
23.....	מסך התחברות -
23.....	מסך הרשמה -
23.....	מסך אימות אימייל (אימות דו שלבי) -
24.....	המסך הראשי -
24.....	מסך הצגת התוכן -
25.....	מבני הנתונים בפרויקט
26.....	פירוט מסד הנתונים המרכזי
26.....	מבנה הטבלה:
26.....	דוגמה לרשומה:
26.....	אובייקט Article
27.....	אובייקט RegularQuery
28.....	אובייקט UrlQuery
28.....	מבנים נוספים
29.....	סקירת חולשות ואיומים
29.....	שכבת האפליקציה
29.....	SQL Injection – עבודה עם בסיס נתונים
29.....	עבודה עם אתרי Web חיצוניים
29.....	תהליך Login – אימות וזיהוי
29.....	התקפת Man In The Middle
30.....	התקפת DOS / DDOS
30.....	קלטות בעייתיים
30.....	שכבת התעבורה (Transport)
30.....	פרוטוקול TCP – לחיצת יד משולשת
30.....	הצפנה:
32.....	הפעלת המערכת – חולשות כלליות
32.....	חולשות אפשריות:
33.....	מימוש הפרויקט
33.....	חלק א' – סקירת מחלקות ומודולים
33.....	מודולים / מחלקות מיובאות
34.....	מחלקות שפותחו בפרויקט
34.....	מחלקה: Article
35.....	מחלקה: DataBase
36.....	מחלקה: Communication

37.....	מחלקה: Login
38.....	מחלקות: RegularQuery, UriQuery
39.....	מחלקה: Save
40.....	קשרי הכלה/ירושה
40.....	בצד הלקוח:
40.....	בצד השרת:
41.....	חלק ב'.
41.....	קטעי קוד רלוונטיים והסברים
41.....	בעיה אלגוריתמית 1: סינון כתבות שאינן ניתנות לקריאה
42.....	בעיה אלגוריתמית 2: שליחת תוכן ארוך בצורה אמינה
43.....	קטע קוד יפה: הצפנה סימטרית מותאמת אישית לכתבות
45.....	חלק ג'.
45.....	בדיקות שתוכננו מראש (בשלב האפיון)
47.....	בדיקות נוספות שבוצעו במהלך הפיתוח
48.....	מדריך למשתמש
48.....	פירוט כלל קבצי המערכת – עץ קבצים
49.....	התקנת המערכת
49.....	הסביבה הנדרשת
49.....	הכלים הדרושים
50.....	מיקומי קבצים
50.....	נתונים התחלתיים
50.....	אופן הפעלת המערכת:
50.....	מריץ את client.py
51.....	התחברות למערכת
52.....	הרשמה למערכת
54.....	חיפוש כתבות לפי נושא או URL וקריאת תוכן הטקסט שלהם
56.....	פתיחת כתבות בשרת פרטי באינטרנט ושמירת הכתבות במחשב
57.....	סיכום אישי / רפלקציה
59.....	ביבליוגרפיה
60.....	נספחים

מבוא

ייזום:

תיאור ראשוני של המערכת

הפרויקט שלי הוא מערכת בשם BarakVPN, שירות VPN ייחודי המאפשר למשתמשים לגלוש באינטרנט בצורה מאובטחת, פרטית וללא הגבלות גישה. המערכת כוללת ממשק לקוח פשוט לשימוש, תקשורת מאובטחת מול שרת מרכזי, וסינון או ניתוב תכנים בהתאם לבקשת המשתמש.

הרציונל לבחירת פרויקט זה נובע מהחשיבות ההולכת וגוברת של פרטיות והגנה על מידע ברשת האינטרנט, במיוחד בעידן שבו מעקב אינטרנטי, וגניבת מידע הם דברים נפוצים מאוד. מטרת הפרויקט היא לאפשר למשתמש לעבור דרך שרת מתווך, כך שהפעולות שיבצע באינטרנט לא ייקשרו לכתובת ה IP שלו.

בחרתי בפרויקט הזה כי הוא משלב תחומים שמעניינים אותי מאוד כמו אבטחת מידע, רשתות ותקשורת בין מחשבים. בנוסף, זהו נושא מאוד רלוונטי כיום, והעיסוק בו מעניק לי ידע וניסיון חשוב.

במהלך פיתוח הפרויקט אני צופה אתגרים טכנולוגיים בתחומים שונים כמו: אבטחת התקשורת בין הלקוח לשרת, בניית ממשק אינטרנטי להצגת התכנים, שליחת קבצים גדולים והתמודדות עם בעיות וחסיונות מצד האינטרנט.

הגדרת הלקוח

המערכת מיועדת לאנשים המעוניינים לבצע חיפושים בצורה מאובטחת, חופשית ופרטית באינטרנט. לקוחות אלו יכולים להיות תלמידים, עובדים, או כל אדם פרטי שרוצה להגן על פרטיותו ברשת ולשמור על סודיות המידע שהוא שולח ומקבל.

הגדרת יעדים

המערכת מיועדת לאפשר למשתמשים לגלוש באינטרנט בצורה מאובטחת, אנונימית וללא הגבלות גישה. המערכת תספק חיבור מוצפן בין המשתמש לבין שרת פרוקסי, אשר יטפל בכל בקשות הגלישה של המשתמש כך שכתובת ה-IP שלו תישאר מוסתרת.

המטרות המרכזיות של המערכת הן:

לאפשר גלישה מאובטחת ואנונימית ברשת על ידי הצפנת התקשורת בין המשתמש לשרת וניתוב התקשורת לפני הגעתה לאינטרנט.

לאפשר עקיפת מגבלות גישה אזוריות או רשתיות באמצעות שימוש בכתובת IP שונה.
לאפשר חוויית משתמש פשוטה ונוחה, כך שכל משתמש יוכל להתחבר ולהשתמש מבלי ידע טכני מוקדם.

בעיות, תועלות וחסכונות

בעיות קיימות:

- אנשים רבים גולשים באינטרנט ללא הגנה, מה שמסכן את פרטיותם.
- אתרי אינטרנט עשויים להגביל גישה לפי מיקום גיאוגרפי או רשת אינטרנט.
- ברשתות ציבוריות קיים סיכון ממשי לגניבת מידע אישי.

תועלות:

- שמירה על פרטיות וזהות המשתמש.
- גישה לתכנים חסומים גיאוגרפית.
- חיבור מאובטח גם דרך רשתות Wi-Fi ציבוריות.
- חסכון בשימוש בכלים חיצוניים בתשלום – פתרון פרטי וחינמי.

שירותים שהמערכת תיתן:

- שירות הצפנה לגלישה מאובטחת.
- אפשרות פתיחת אתרים באתר פרטי.
- ממשק API להזרמת תוכן מהשרת ללקוח.

סקירת פתרונות קיימים

קיימות מערכות כמו NordVPN, ExpressVPN ו-ProtonVPN המציעות שירות דומה. אך לרוב מדובר בשירותים מסחריים בתשלום שאינם נותנים שליטה מלאה על השרת או על פרטי ההצפנה. בנוסף המשתמש אינו יודע מה הולך מאחורי הקלעים בשרתים שלהם ואינו יכול להיות בטוח שהמידע שלו באמת מאובטח. הפתרון שמוצע בפרויקט זה הוא פתוח, גמיש ונבנה מאפס, מה שמאפשר התאמה מלאה לצרכים אישיים ושקיפות מלאה עם המשתמש.

סקירת טכנולוגית הפרויקט

הפרויקט עושה שימוש בטכנולוגיות קיימות כמו:

- שפת Python
- תקשורת TCP עם הצפנת RSA וצופן קיסר.
- פרוטוקולי רשת
- אפליקציית Flask עבור ממשק אינטרנטי להצגת תוכן.

הטכנולוגיות אינן חדשות, אך השילוב ביניהן כן, יצירת VPN פרטי ייחודי ונוח לשימוש.

תיחום הפרויקט

המערכת תטפל בתחומים הבאים:

- אבטחת תקשורת בין לקוח לשרת.
- יצירת שכבת תיווך לשימוש ברשת בצורה אנונימית.
- ניהול משתמשים ואימותם.

המערכת לא תטפל בתחומים הבאים:

- אינטגרציה מלאה עם מערכת ההפעלה.
- סינון תכנים בחיפוש.

פירוט תיאור המערכת (אפיון):

תיאור מפורט יותר של המערכת

המערכת תקבל את מה שהלקוח רוצה לחפש, תחפש אותו ברשת ותחזיר ללקוח את התוצאות של חיפוש, ללקוח תנתן אפשרות לבחור בין מספר תוצאות שונות מהאינטרנט אשר הוא יוכל לראות את תוכן הטקסט שלהם ואף לפתוח את הדף הראשוני של האתר בשרת פרטי.

המערכת תאפשר חיפוש באינטרנט, הצגת תוכנם ופתיחת הדפים בצורה אנונימית לגמרי.

פירוט היכולות שהיא תעניק לכל סוג משתמש במערכת

המערכת תיתן למשתמש אפשרות:

- לבצע חיפוש ברשת בצורה אנונימית.
- לקרוא את התכנים של האתרים אותם חיפש.
- לפתוח את הדף של האתר באתר אינטרנט פרטי.

טבלת הבדיקות המתוכננות

תרחיש בדיקה	מה בודקים	קלט לדוגמה	תוצאה צפויה
הרשמה עם שם משתמש חוקי, סיסמה חוקית ואימייל תקני	שהמערכת מאפשרת הרשמה	username=barak email=barak@gmail.com password = "BarakVPN1"	מתקבלת תשובת "True"
הרשמה עם שם משתמש המכיל קללה	סינון מילות גנאי	username=racistguy123	מתקבלת תשובת "forbidden words"
הרשמה עם אימייל לא תקני	בדיקת פורמט אימייל	email=@gmail.com (אין כתובת)	מתקבלת תשובת "problem"
התחברות עם סיסמה נכונה	שהמשתמש מזוהה ונשלח קוד אימות	שם משתמש + סיסמה חוקיים	מתקבלת תשובת "True/code/mail"
התחברות עם סיסמה שגויה	אימות נכשל	סיסמה שגויה	מתקבלת תשובת "False"
חיפוש רגיל עם נושא תקני	החזרת כתבות רלוונטיות	"query="War in Ukraine"	רשימת כתבות באורך < 0
חיפוש עם כתובת URL ישירה	שהמערכת מחזירה כתבה ספציפית	query="https://www.baltimoreravens.com/news/derrick-henry-ravens-two-year-extension-contract-2025"	כתבה בודדת חוזרת
חיפוש ריק	שלא נשלחת בקשה לשרת	" " = query	אין תוצאות חוזרות
פתיחת כתבה בדפדפן	ששרת Flask מופעל ומציג את התוכן	לחיצה על כפתור "Open in browser"	נפתח דפדפן עם כתבה

לא תופיע ברשימת הכתבות	כתובת בעייתית	שהכתבה נזרקת לפני שנשלחת ללקוח	כתבה שחסומה או מכילה CAPTCHA
תוצג הודעת שגיאה "incorrect code"	קוד שגוי	שהמערכת לא מאמתת כניסה	קוד אימות שגוי (ב-Login)

תכנון וניהול לו"ז זמנים לפיתוח המערכת:

1.1 התחלת חשיבה לגבי נושא הפרויקט

19.1 בחירת הנושא

20.1-31.2 בדיקת היתכנות של הפרויקט ופיתוח גרסה ראשונית.

1.3 הצגה ראשונית

1.3-1.4 שיפור הגרסה הראשונית, הוספת פיצ'רים ותקשורת

1.4 הצגה שניה

1.4-1.5 שיפור הפרויקט, תיקון באגים, טיפול במקרי קצה ושגיאות, הצפנה

1.5 הצגה שלישית

1.5-16.5 בדיקות אחרונות, עוד טיפול בשגיאות ומקרי קצה

16.5-24.4 עבודה על נראות ושיפור האבטחה

ניהול הסיכונים בפרויקט והדרכים להתמודדות איתם

סיכון	תיאור הסיכון	תכנון התמודדות	מה בוצע בפועל
בעיות תקשורת בין לקוח לשרת	קושי בהעברת מידע רב, כמו התוכן של האתרים.	שימוש בפרוטוקול TCP עם תכנון לשליחת אורך הודעה מראש (8 בתים), כך שהלקוח יודע כמה נתונים לקרוא.	השתמשתי בספריית pickle על מנת לשלוח את כל המידע בפעם אחת, ביחד עם הגודל של המידע על מנת שהלקוח ידע כמה לקרוא.
כשל בהצפנת נתונים	הצפנה לא תקינה עלולה לגרום לכך ששרת לא יוכל להבין את הבקשה.	קושי בשימוש בשיטות הצפנה מכיוון שלא ידעתי את הנושא הזה טוב	ההצפנה עובדת עם מפתחות קבועים ונבדקה ידנית בשלבי הפיתוח.
חוסר ידע בטכנולוגית flask ודפי html	היה לי ידע מוקדם מצומצם מאוד בתחומים אלה	למידה עצמית בבית ופיתוח יכולת.	עשיתי מחקר ולמידה לפני כל שלב.

תיאור תחום הידע - פרק מילולי

יכולות המשתמש

- התחברות למערכת
- הרשמה מאובטחת למערכת
- ביצוע חיפושים רגילים בצורה אנונימית
- ביצוע חיפוש לפי קישור בצורה אנונימית
- פתיחת קישור לצפייה בדפדפן מקומי בצורה אנונימית
- שמירת תוכן הכתבות בקובץ טקסט

יכולות בצד שרת

- טיפול בלקוחות במקביל באמצעות threads
- זיהוי עומסים
- שליחה/קבלה של נתונים רבים
- פונקציית טעינת Flask דינמית
- טיפול בבקשות הלקוח
- קבלת מידע מהאינטרנט

פירוט:

• התחברות למערכת -

מהות: התחברות למשתמש שלך אימות דו שלבי על מנת להגביר אבטחה.

אוסף יכולות נדרשות:

ממשק משתמש - מסך התחברות המקושר למסך ההרשמה ומציג גם שגיאות.

הצפנת הנתונים - הנתונים של המשתמש מוצפנים לפני השליחה לשרת, שם המשתמש באמצעות RSA והסיסמה באמצעות Hash.

שליחת נתונים לשרת - נתוני ההתחברות נשלחים לשרת.

פיענוח הנתונים - השרת מפענח את הנתונים בעזרת מפתחות קבועים בקוד של המערכת.

בדיקה האם הנתונים מתאימים למשתמש במערכת - במידה והנתונים אינם תואמים, השרת מחזיר ללקוח שאין משתמש כזה

קבלת תשובה מהשרת

הצגת התשובה למשתמש - מוצגת ללקוח הסיבה לכך שאי אפשר להתחבר למשתמש שלו.

שליחת הודעה במייל - במידה והנתונים תואמים למשתמש במערכת, השרת שולח הודעה במייל שמכילה קוד אימות לכתובת שרשומה עם הלקוח.

שליחת נתונים בחזרה ללקוח - השרת מחזיר ללקוח הודעה שהוא הצליח להתחבר, ביחד עם קוד האימות אותו הלקוח צריך להזין, בצורה מוצפנת עם RSA.

קבלת תשובה מהשרת

פענוח - הלקוח מפענח את הנתונים בעזרת מפתחות קבועים בקוד של המערכת.

פתיחת מסך חדש - נפתח מסך חדש שבו כתובה כתובת המייל אליה נשלח הקוד ושדה ריק שבו המשתמש צריך למלא את הקוד, במידה והוא ממלא נכון, החלון נסגר

ונפתח החלון הראשי של האפליקציה, במידה והוא עונה לא נכון מוצגת לו הודעה שהקוד לא נכון.

אובייקטים נחוצים: : ממשק משתמש, הצפנה/פענוח, תקשורת, בסיס נתונים, ספרייה לשליחת מיילים.

● הרשמה מאובטחת למערכת

מהות: הרשמות לאתר על ידי סיסמה, אימייל ושם משתמש.

אוסף יכולות נדרשות:

ממשק משתמש - מסך הרשמה המקושר למסך ההתחברות ומציג גם שגיאות.

בדיקת דרישות סיסמה בלקוח - הסיסמה נבדקת אם היא מכילה את כל הדרישות של סיסמה חזקה לפני שהיא נשלחת לשרת על מנת שיהיה אפשר להצפין אותה בצורה הכי טובה.

הצפנת הנתונים - הנתונים של המשתמש מוצפנים לפני השליחה לשרת, האימייל ושם המשתמש באמצעות RSA והסיסמה באמצעות Hash.

שליחת נתונים לשרת - נתוני ההרשמה נשלחים לשרת.

פיענוח הנתונים - השרת מפענח את הנתונים בעזרת מפתחות קבועים בקוד של המערכת.

בדיקת תקינות שם המשתמש והאימייל - שם המשתמש והאימייל נבדקים אם הם תקינים, במידה והם לא השרת שולח ללקוח הודעה מותאמת בהתאם לשדה שלא תקין.

הוספה למסד המידע - במידה והשם משתמש והאימייל תקינים, השרת בודק האם לפחות אחד מהם כבר קיים במסד המידע ובמידה שכן הוא מחזיר ללקוח הודעה שאי אפשר ליצור חשבון, אם הם לא קיימים הוא מצרף אותם ואת הסיסמה למסד המידע ושולח הודעה לקליינט שהוא התחבר בהצלחה.

קבלת תשובה מהשרת

הצגת התשובה למשתמש - במידה ולא היה ניתן ליצור משתמש, מוצג ללקוח הסיבה לכך, במידה ונוצר המשתמש הלקוח מועבר למסך הראשי של האפליקציה.

אובייקטים נחוצים: : ממשק משתמש, הצפנה/פענוח, תקשורת, בסיס נתונים.

● ביצוע חיפוש רגילים

מהות: קבלת התוצאות של החיפוש בצורה אנונימית וקריאת התוכן שלהן.

אוסף יכולות נדרשות:

ממשק משתמש - מסך חיפוש המציג גם שגיאות.

הצפנת הנתונים - המשפט אותו המשתמש חיפש והכתבות שהלקוח כבר חיפש מוצפנים באמצעות RSA.

שליחת נתונים לשרת

פיענוח הנתונים - השרת מפענח את הנתונים בעזרת מפתחות קבועים בקוד של המערכת.

חיפוש באינטרנט - השרת מנסה לחפש בגוגל את המשפט אותו הלקוח ביקש, במידה וגוגל לא פועל הוא מנסה שוב ב"duckduckgo"

שמירת מידע על כל תוצאה והצפנתו - השרת יוצר רשימה של אובייקטים מסוג article ולכל אובייקט יש את המאפיינים של כל כתבה מוצפנים לקראת שליחה, זה קורה במקביל באמצעות multiprocessing על מנת לחסוך בזמן.

בדיקה אם קיים כבר - השרת בודק האם הכתבה שמצא כבר קיימת ברשימת הכתבות שהלקוח כבר חיפש, אם לא הוא מצרף אותה לרשימה החדשה.

שליחת רשימת הכתבות ללקוח

פיענוח המידע - הלקוח מפענח את המידע שקיבל מהשרת.

הצגת המידע - מופיעות במסך הלקוח כותרות הכתבות בצורות כפתורים שכאשר לוחצים עליהם מופיע התוכן בטקסט של כל כתבה. הכתבות מוצגות 5 בכל עמוד וכמות העמודים משתנה לפי כמות הכתבות שהוא קיבל.

אובייקטים נחוצים: : ממשק משתמש, הצפנה/פיענוח, תקשורת, googlesearch-python, duckduckgo_search, requests

● ביצוע חיפוש לפי קישור

מהות: לקבל את התוכן טקסט של האתר ואפשרות לפתוח את הקישור הרצוי בצורה אנונימית.

אוסף יכולות נדרשות:

ממשק משתמש - מסך חיפוש המציג גם שגיאות.

הצפנת הנתונים - הקישור אותו המשתמש חיפש והכתבות שהלקוח כבר חיפש מוצפנים באמצעות RSA.

שליחת נתונים לשרת

פיענוח הנתונים - השרת מפענח את הנתונים בעזרת מפתחות קבועים בקוד של המערכת.

שמירת מידע על האתר והצפנתו - השרת יוצר רשימה באורך 1 של אובייקטים מסוג article ולכל אובייקט יש את המאפיינים של כל כתבה מוצפנים לקראת שליחה.

שליחת רשימת הכתבות ללקוח

פיענוח המידע - הלקוח מפענח את המידע שקיבל מהשרת.

הצגת המידע - מופיעות במסך הלקוח כותרות הכתבות בצורות כפתורים שכאשר לוחצים עליהם מופיע התוכן בטקסט של כל כתבה. הכתבות מוצגות 5 בכל עמוד וכמות העמודים משתנה לפי כמות הכתבות שהוא קיבל.

אובייקטים נחוצים: ממשק משתמש, הצפנה/פיענוח, תקשורת, requests.

● פתיחת קישור לצפייה בדפדפן מקומי

מהות: לפתוח את האתר בדף אינטרנט על מנת לראות את תוכנו בצורה יותר ויזואלית, ומאפשר לראות תמונות.

אוסף יכולות נדרשות:

ממשק משתמש - כפתור "open in browser" שמוצג במסך שמציג את תוכן הכתבה.

פתיחת אפליקצית flask - אפליקצית flask חדשה נפתחת בצד הלקוח ושולחת בקשה לשרת מספר פעמים על מנת לקבל את כל הנתונים של הpaths השונים שהאתר צריך על מנת להיפתח.

שליחת נתונים לשרת - הלקוח שולח את בקשה לשרת בהתחלה עם הpath ההתחלתי ואז עם הpaths השונים שהאתר מבקש.

קבלת תוכן מהאינטרנט - השרת מבצע בקשת get לכתובת שהתקבלה בעזרת פפריית requests.

שליחת התוכן בחזרה ללקוח – השרת משיג את התוכן, קוד הסטטוס וסוג הקובץ (Content-Type) ושולח מחרוזת אחת הכוללת את שלושתם ללקוח.

קבלת תוכן מהשרת – הלקוח מקבל את המחרוזת, מחלץ ממנה את התוכן, הקוד וסוג הקובץ.

הצגת התוכן בדפדפן – Flask מחזיר את התוכן כתשובת HTTP לדפדפן, כולל הסטטוס וסוג התוכן. התוצאה מוצגת למשתמש כאילו הגיע מאתר רגיל.

אובייקטים נחוצים: ממשק משתמש, תקשורת, Flask, requests.

● שמירת תוכן הכתבות בקובץ טקסט

מהות: לשמור את התוכן של כתבות שהמשתמש אוהב במחשב.

אוסף יכולות נדרשות:

ממשק משתמש – כפתור "Save this article" שמוצג במסך שמציג את תוכן הכתבה.

יצירת קבצים - יצירת קובץ טקסט חדש אם לא קיים כבר, ואיתחול השם שלו לכותרת הכתבה.

שמירת תוכן הכתבות - שמירת הקישור לכתבה ואת תוכן הכתבות בקובץ הטקסט.

אובייקטים נחוצים: os.

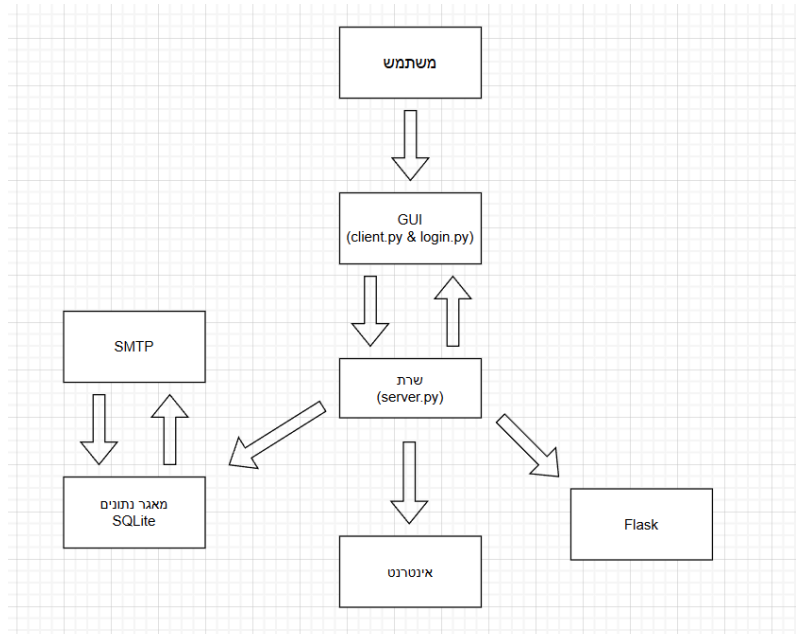
מבנה / ארכיטקטורה של הפרויקט

BarakVPN היא מערכת מבוססת לקוח ושרת, שמטרתה לאפשר למשתמשים לחפש כתבות ממנועי חיפוש (Google, DuckDuckGo) בצורה אנונימית, ולהציג אותן בסביבה מאובטחת ומוצפנת. המערכת כוללת מנגנוני אבטחה כמו הצפנת RSA, הגבלת משתמשים בו זמניים, ואימות דו-שלבי באימייל.

תיאור החומרה – רכיבים שונים והקשרים ביניהם:

רכיבי המערכת:

שרת מרכזי	שרת שמאזין לחיבורים ומטפל בבקשות לקוח. מבצע חיפוש, איסוף תוכן, בדיקות התחברות ורישום.
מוד מידע SQLite	מאחסן את פרטי המשתמשים (שם משתמש, סיסמה מוצפנת, אימייל).
ממשק משתמש GUI	אפליקציית פייתון שמבוססת על customtkinter. מציגה תוצאות, מאפשרת שימוש בפונקציות המערכת.
שרת Flask	מופעל בצד הלקוח לצפייה בתוכן דינמי של כתבות בדפדפן ("127.0.0.1:5000").
אינטרנט	תקשורת עם מנועי חיפוש חיצוניים (Google/DuckDuckGo) לקבלת תוצאות.
SMTP	נשלחות באמצעותו הודעות אימות לאימות דו-שלבי בעת התחברות למערכת.



הטכנולוגיה הרלוונטית:

שפת תכנות:

הפרויקט נבנה בשפת **Python**.

מערכת הפעלה:

הפיתוח והבדיקות בוצעו על מערכת הפעלה **Windows**, אך המערכת תואמת ברובה גם למערכות הפעלה אחרות וניתנת להעברה בקלות לסביבת שרת.

טכנולוגיות תקשורת:

- **Socket TCP** – לתקשורת יציבה בין הלקוח לשרת. נבנה פרוטוקול מותאם אישית להעברת שאלות ותוכן.
- **Pickle** – לעיבוד ושליחה של אובייקטים בצורה בינארית בין רכיבי המערכת.
- **RSA** – להצפנת הנתונים (כגון סיסמאות ושאלות) טרם שליחתם.

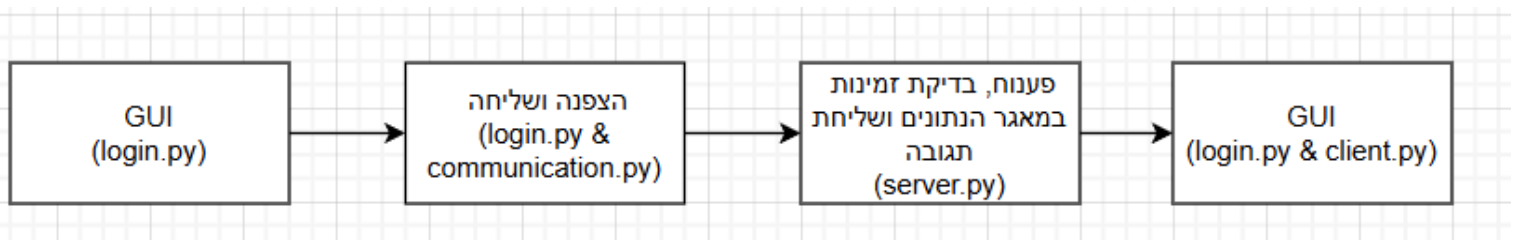
תחומי עניין טכנולוגיים בפרויקט:

- **אבטחת מידע** – יישום RSA להצפנת מידע תוך כדי זרימת העבודה.
- **רשתות מחשבים** – הקמת תקשורת אמינה בין רכיבי מערכת מרוחקים.
- **פיתוח מערכות מבזרות** – לקוח, שרת ותצוגה (Flask) פועלים כחלקים נפרדים המתואמים ביניהם.
- **פיתוח Web מקומי** – שימוש ב-Flask להצגת תוכן בצורה מאובטחת בדפדפן.

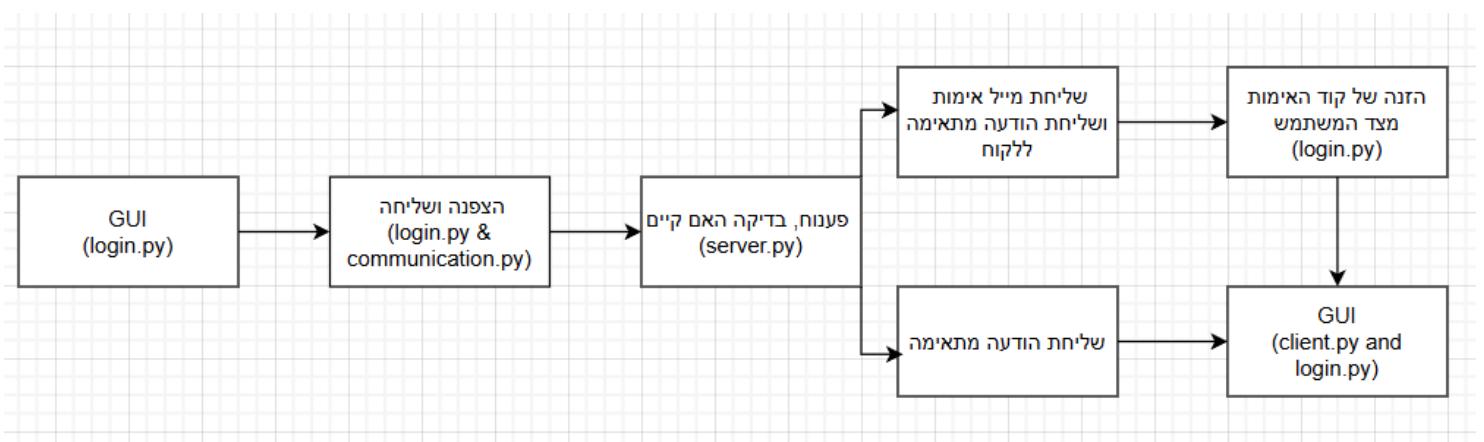
זרימת המידע במערכת

1. רישום משתמש (Sign Up):

- המשתמש מזין שם משתמש, אימייל וסיסמה.
- הנתונים מוצפנים ב-RSA בצד הלקוח ונשלחים לשרת.
- השרת מפענח, בודק אם המשתמש כבר קיים ומחזיר תשובה מתאימה.
- אם הכל תקין, פרטי המשתמש מאוחסנים במסד הנתונים.

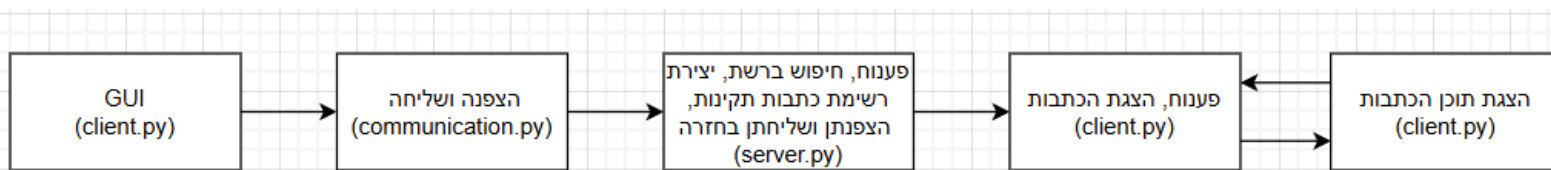


- השרת בודק אם המשתמש קיים.
- אם תקין, נשלח קוד אימות במייל.
- המשתמש מזין את הקוד לאימות.



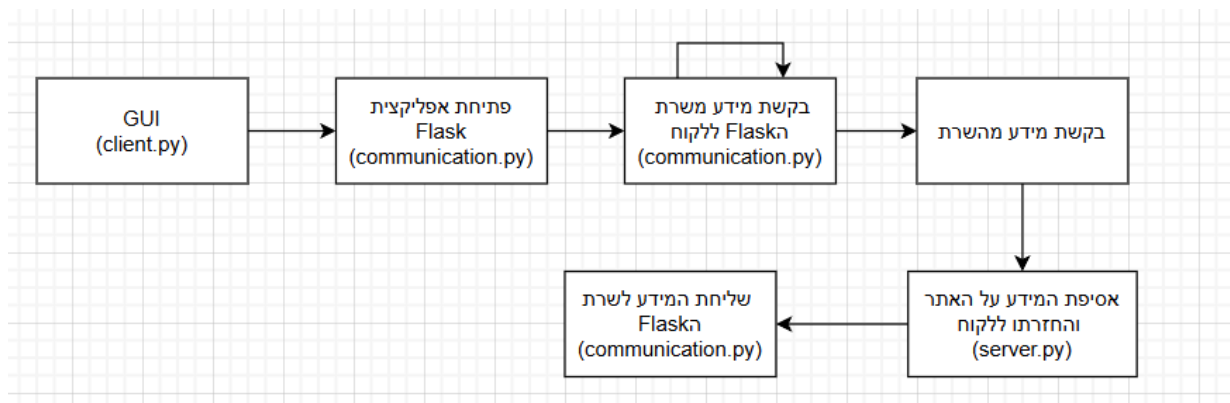
3. חיפוש כתבות:

- המשתמש מזין נושא חיפוש.
- הלקוח שולח בקשה מוצפנת לשרת.
- השרת מבצע חיפוש (Google/duckduckgo), בונה רשימת כתבות תקינות (Articles), מצפין ושולח חזרה.
- הלקוח מפענח ומציג.
- אפשרות לקריאת תוכן הכתבות



4. צפייה בכתבה (Open in browser):

- המשתמש לוחץ על פתיחת הכתבה.
- הלקוח מפעיל שרת Flask מקומי.
- שרת Flask שואל את המערכת על תוכן האתר.
- השרת מחזיר את תוכן העמוד המוצפן בשביל כל בקשה של שרת Flask.
- Flask מציג את הדף בדפדפן.



תיאור האלגוריתמים מרכזיים בפרויקט:

בעיה אלגוריתמית 1: השגת קישורים מהרשת

הבעיה: גוגל חוסם את מחשב השרת מלהשתמש בו לאחר כמה פעמים.

- מציאת דרך להשיג קישורים גם כאשר גוגל חוסם את המחשב.

פתרונות קיימים:

- שימוש במפתח API רשמי של גוגל.
- שימוש במנוע חיפוש אחר.

הפתרון שלי:

- שימוש במנוע חיפוש גוגל עד שהוא חוסם את המחשב, ולאחר שהוא חוסם את המחשב מנוע החיפוש עובר להיות duckduckgo.
- מתבצע בצד השרת לפני שליחת כתבות ללקוח.

נימוק הבחירה:

- יעיל מאוד.
- חינם
- קל למימוש במסגרת הפרויקט.

בעיה אלגוריתמית 2: סינון כתבות שאינן ניתנות לקריאה

הבעיה: כיצד ניתן לזהות ולסנן כתבות שהשרת הצליח לאחזר את כתובת ה URL שלהן, אך לא ניתן לחלץ מהן תוכן קריא?

אתגרים:

- לא כל כתובת שנמצאת במנוע חיפוש מובילה לאתר תקני.
- חלק מהכתבות מוגנות על ידי חומת אש, CAPTCHA, או דורשות הרשאות.
- אם לא מבוצעת בדיקה – המשתמש מקבל כתבות לא שמישות.

פתרונות קיימים:

- ביצוע Headless-Browser כמו Selenium שמתנהג כמו מנוע חיפוש אמיתי.

הפתרון שנבחר בפרויקט:

- בעת איחזור תוכן הכתבה נבדקים מספר תנאים:
- האם קיימת כותרת `<title>?`
- האם קיימים אלמנטים של תוכן (`<h1>`, `<p>` וכו')?
- האם סטטוס ה-HTTP שונה מ-200?
- האם יש מילים חשודות בתוכן כמו: "captcha", "access denied", "robot", "check", "blocked" וכו'.
- אם אחד התנאים מתקיים – הכתבה נזרקת ולא נשלחת ללקוח.

נימוק הבחירה:

- פתרון מהיר, וללא תלות בספריות כבדות כמו Selenium.
- מאפשר ביצוע סינון אמין בשרת טרם שליחה.
- הכתבות שנשלחות ללקוח הן רק כאלה שעברו סינון.

בעיה אלגוריתמית 3: שליחת תוכן ארוך מהשרת ללקוח

הבעיה: כיצד ניתן לשלוח תכנים באורך של מאות או אלפי תווים (כתבות מלאות מהאינטרנט) מהשרת ללקוח בצורה אמינה, מבלי שהמידע ייחתך או ייפגם?

אתגרים:

- `socket.recv()` לא מבטיח קבלה של כל המידע בהודעה אחת.
- בעת העברת מידע גדול (מעל 4096 בתים), נדרשת קריאה חוזרת עד להשלמת כל המידע.
- אם התוכן חלקי – תהליך הפענוח (Pickle) ייכשל או שהכתבה תוצג חלקית.

פתרונות קיימים:

- שימוש בפרוטוקולי HTTP מלאים עם תמיכה בתוכן משתנה.
- שימוש בספריות מתקדמות כמו requests, או פרוטוקולי RPC.

הפתרון שנבחר בפרויקט:

- השרת שולח את אורך הנתונים (8 בתים) לפני שליחת התוכן עצמו.
- בצד הלקוח, מתבצע לולאת recv() עד שאורך המידע שהתקבל שווה לאורך שצוין מראש.
- רק לאחר קבלת כל המידע, הוא נשלח לשחזור באמצעות pickle.loads().

נימוק הבחירה:

- פתרון פשוט, נוח, ויעיל שאינו דורש פרוטוקולים נוספים כמו HTTP או RPC.
- שליטה על כמות הנתונים.
- עמיד בתקשורת איטית או מקוטעת.

תיאור סביבת הפיתוח

כלי הפיתוח הדרושים לפיתוח המערכת:

- שפת תכנות: Python 3.10 ומעלה.
- סביבת פיתוח: pycharm – עורך קוד קל ונוח.
- דיסק און קי: לשמירת גרסאות.

סביבה וכלים לבדיקות:

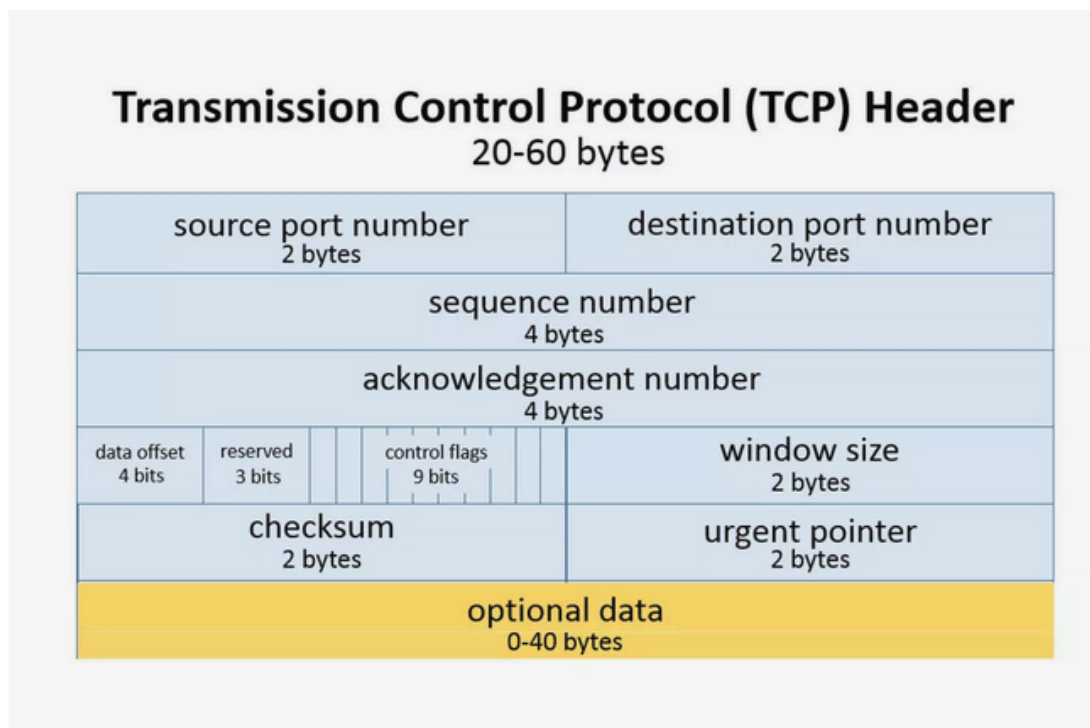
- סימולציה מקומית: הפעלת שרת ולקוח באותו המחשב.
- הדפסות למסך לזיהוי בעיות.
- שימוש ב DBbrowser על מנת לבדוק את תקינות פרטי המשתמש.

תיאור פרוטוקול התקשורת

תיאור מילולי של פרוטוקול התקשורת:

- פרוטוקול **TCP (Transmission Control Protocol)** הוא פרוטוקול תקשורת אמין מבוסס חיבור (Connection-oriented), הפועל בשכבת התעבורה של מודל ה-TCP/IP. נועד לאפשר העברת נתונים אמינה בין שני מחשבים ברשת, תוך הבטחה שכל החבילות יועברו שלמות, בסדר הנכון וללא כפילויות. בעת פתיחת חיבור TCP, מתבצע תהליך שנקרא "לחיצת יד תלת-שלבית" (Three-Way Handshake) שבאמצעותו שני הצדדים מסנכרנים ביניהם ומקימים חיבור תקשורת יציב. TCP כולל מנגנוני בקרה כגון בדיקת שגיאות, אישורי קבלה (ACK), וויסות זרימה, וניהול עומסים – מה שהופך אותו למתאים במיוחד ליישומים הדורשים אמינות.

דוגמה של פקטת TCP:



סוגי ההודעות הזורמות במערכת:

- הודעות פקודה: מגיעות מהלקוח לשרת, מתחילות בפקודה ("uRI" "loGIn" "siGnUp", "oPeN") ואחריהן משתנים מופרדים על ידי תוים שאותם בוודאות אין בתוך המשתנים, הודעות אלו מפעילות פעולה אחרת בשרת ומקבלות תשובה בהתאם.

- הודעות תשובה: מגיעות מהשרת ללקוח, לרוב מכילות רק תשובה בטקסט אך לעתים גם נשלחים משתנים דרכן, לדוגמה: "True", "False", "email problem", ועוד...

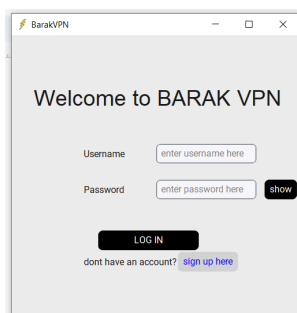
הודעות אלו נשלחות כתשובות להודעות פקודה של הלקוח והלקוח מציג תשובה למשתמש בהתאם.

- הודעות בינאריות (Pickle): נשלחות גם מהלקוח לשרת וגם מהשרת ללקוח, תלוי בסיטואציה, מועברות עם אורך מקדים של 8 בתים, ולאחר מכן הנתונים עצמם, הודעות אלו משמשות לשליחת תכנים ארוכים כמו עצמים של כתבות.

תיאור מסכי המערכת:

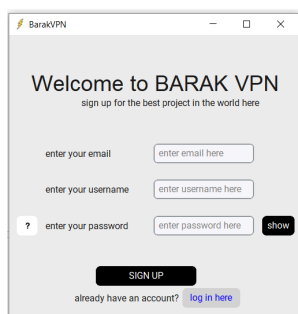
מסך התחברות -

המסך הראשון במערכת הוא מסך הרשמה (log in) שבו לקוחות שכבר יש להם משתמשים, יש אפשרות או להתחבר למשתמש או לעבור למסך ההרשמה.



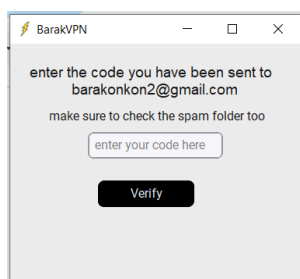
מסך הרשמה -

מסך שבו נרשמים למערכת, מתאים ללקוחות חדשים, יש אפשרות או להרשם או לעבור למסך ההתחברות.



מסך אימות אימייל (אימות דו שלבי) -

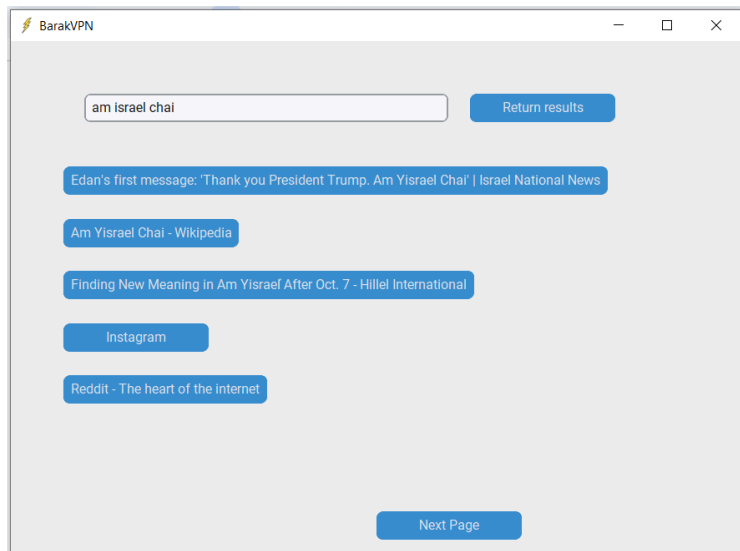
מסך שמטרתו לאמת את המשתמש על ידי האימייל איתו נרשם למערכת, למסך הזה הלקוח מועבר לאחר שניסה להתחבר למשתמש שלו וממנו מועברים למסך הראשי במידה ומכניסים את הקוד הנכון.



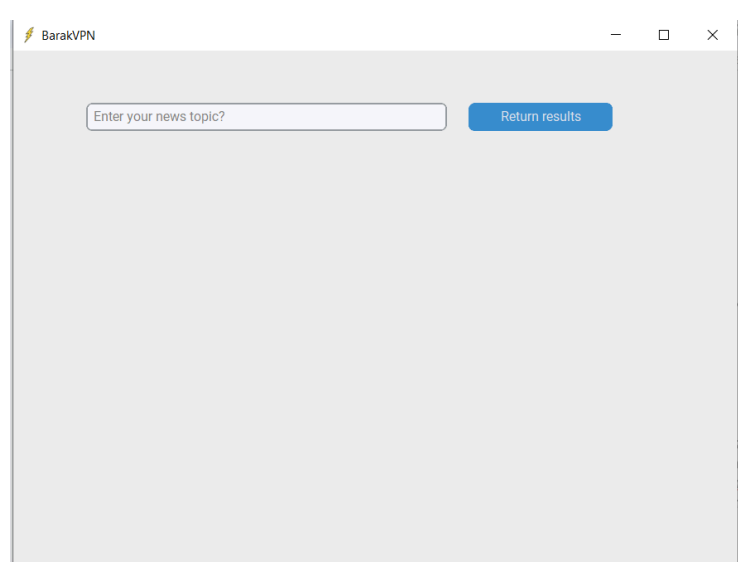
המסך הראשי -

במסך זה הלקוח מכניס את מה שברצונו לחפש באינטרנט, מידע זה נשלח לשרת והשרת מחזיר לו את התוצאות של החיפוש שלו, התוצאות מוצגות גם במסך זה, למסך זה יש כמות עמודים שמשתנה בהתאם לכמות הכתבות שמופיעות, חמש כתבות בכל עמוד. אל המסך הזה מגיעים לאחר שמסיימים את תהליך ההתחברות/הרשמה.

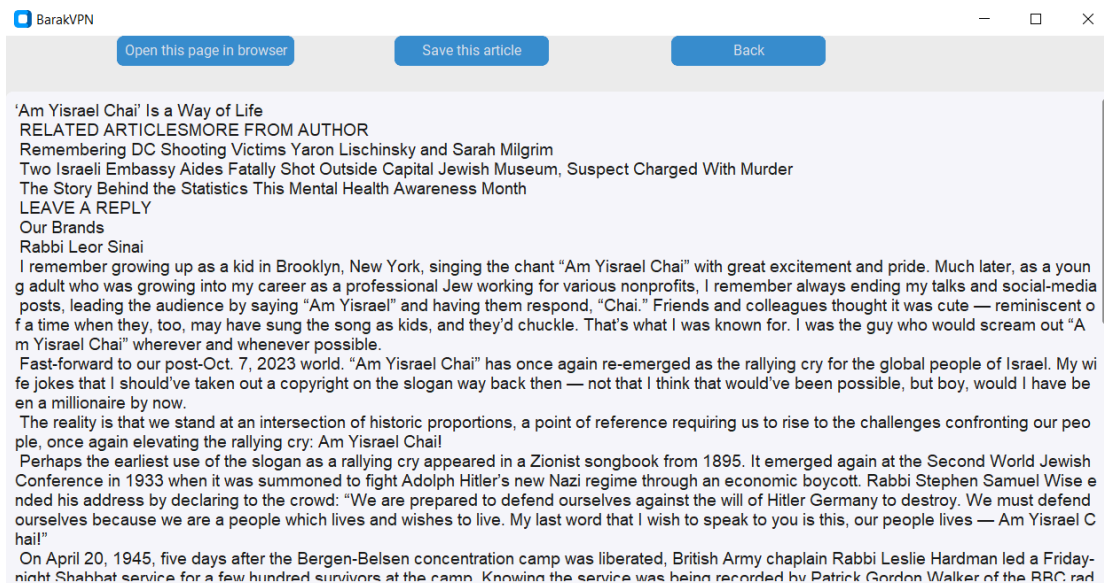
אחרי ביצוע חיפוש

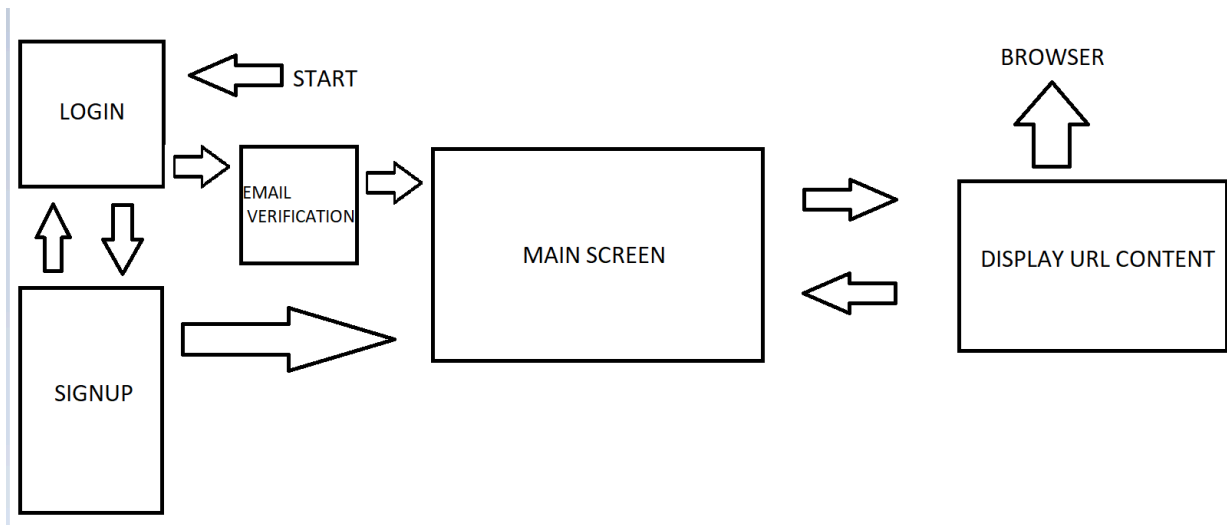


לפני ביצוע חיפוש



מסך זה מציג ללקוח את התוכן טקסט של האתר אותו בחרת לפתוח במסך הראשי, במסך זה יש גם אפשרות לפתוח את התוכן של האתר בדפדפן באפליקציית flask. אל מסך זה מגיעים לאחר שלוחצים על אחת מהכתבות שהוצגו במסך הראשי.





תיאור מבני הנתונים

מבני הנתונים בפרויקט

המערכת משתמשת בכמה סוגים של מבני נתונים:

סוג מבנה נתונים	שימוש בפרויקט	פירוט
מסד נתונים SQLite	שמירת נתוני משתמשים (שם, סיסמה, אימייל)	נשמר בקובץ credentials.db
אובייקטים מסוג Article	שמירת כתבות מאוחזרות מהאינטרנט במבנה מונחה עצמים	כוללים את המאפיינים של הכתבות
אובייקט מסוג RegularQuery	משמש ליצור הודעת פקודה לשרת כאשר צריך ליצור הודעה עם חיפוש רגיל, יורש מ־QueryType	מכיל את מספר הכתבות הקיימות ואת משפט החיפוש ופונקציית ליצירת פקודה
אובייקט מסוג UrlQuery	משמש ליצור הודעת פקודה לשרת כאשר צריך ליצור הודעה עם חיפוש של קישור, יורש מ־QueryType	מכיל את מספר הכתבות הקיימות ואת משפט החיפוש ופונקציית ליצירת פקודה
רשימות ומילונים של פייתון	משמשים לשמירת מידע באופן זמני	לדוגמה, titles, articles ועוד...

פירוט מסד הנתונים המרכזי

- שם המסד: credentials.db
- טבלת נתונים יחידה: logins

מבנה הטבלה:

שם	טיפוס	דוגמה לערך	הסבר
username	TEXT	"barakuser"	שם משתמש ייחודי
password	TEXT	"5e884898...f843"	סיסמה אחרי הצפנת האש
email	TEXT	"user@gmail.com"	כתובת אימייל תקנית

- המפתח הראשי: השדה username

- מגבלות:
 - username – ייחודי
 - email – ייחודי

דוגמה לרשומה:

username: barakvpn
password: e3afed0047b08059d0fada10f400c1e5f46ef...
email: barakvpnoofficial@gmail.com

אובייקט Article

הכתבות מאוחזרות מהאינטרנט ומאוחסנות כאובייקטים מהמחלקה Article.

שדות עיקריים:

שדה	טיפוס	תיאור
url	str	קישור ישיר לכתבה, משמש לאסיפת נתונים
title	list / str	כותרת הכתבה, חלק מהזמן מוצפנת ב-RSA, חלק מהזמן טקסט
content	str	תוכן הכתבה, לאחר עיבוד והצפנה

code	list / str	קוד ההצפנה של תוכן הכתבה, חלק מהזמן מוצפן ב-RSA, חלק מהזמן טקסט
------	------------	---

פעולות:

פעולה	תיאור
GetInfo()	מבקש מהאתר את הפרטים שלו ומכניס לכל המשתנים של האובייקט את הערך שלהם, בנוסף מתבצעת בדיקה האם הכתבה ניתנת לשימוש ולא נחסמת על ידי האתר ומוחזר ערך בוליאני בהתאם.
RSAencrypt(pk, plaintext)	מצפין את הטקסט שהוא מקבל ב-RSA לפי המפתח אותו מקבל

אובייקט RegularQuery

משמש ליצירת בקשת פקודה לשרת כאשר המשתמש ביצע חיפוש רגיל (לא לפי קישור), לדוגמה: "am israel chai"

שדות עיקריים:

שדה	טיפוס	תיאור
query	str	מחרוזת החיפוש המקורית שהוזנה ע"י המשתמש
start	int	מספר הכתבות שכבר התקבלו, כדי להתחיל חיפוש ממקום מסוים

פעולות:

פעולה	תיאור
setQuery(query)	מגדירה את מחרוזת החיפוש
format_for_sending()	מחזירה את הפקודה בפורמט שהשרת מצפה לו, b'uRI/search/query/start'

אובייקט UriQuery

משמש ליצירת בקשת פקודה לשרת כאשר המשתמש ביצע חיפוש לפי קישור ישיר (URL), לדוגמה: "https://google.com"

שדות עיקריים:

שדה	טיפוס	תיאור
query	str	כתובת ה-URL שהוזנה על ידי המשתמש
start	int	מספר הכתובות שכבר הוצגו (0 בחיפוש URL כי מדובר בכתבה בודדת)

פעולות:

פעולה	תיאור
setQuery(query)	מגדירה את הכתובת
format_for_sending()	מחזירה את הפקודה בפורמט שהשרת מצפה לו, b'URL/url/query/start'

שני המחלקות יורשות את המחלקה QueryType ומממשות את אותו ממשק ליצירת הודעת פקודה מותאמת לשרת.

מבנים נוספים

- `titles = [title1, title2, ...]`
משמשת למעקב אחרי כותרות שנשלחו ללקוח כדי למנוע כפילויות.
- `articles = [Article, Article, ...]`
רשימת כתבות שמועברות ללקוח מהשרת כ-pickle.
- `dictionary/list פנימיים בשרת ולקוח`
משמשים לעיבוד של טקסטים, ניתוח כתובות, אחסון זמני.

סקירת חולשות ואיומים

שכבת האפליקציה

SQL Injection – עבודה עם בסיס נתונים

- **בעיה:** משתמש זדוני יכול להזין קלט עם שאילתות SQL מזיקות.

מניעה: שימוש בפקודות מוכנות ללא שילוב משתנים בסטרינג הפקודה.

עבודה עם אתרי Web חיצוניים

- **בעיה:** אתרים עלולים להחזיר תוכן בעייתי (CAPTCHA, חומת אש, חסימת בוטים).

- **פתרון:** אלגוריתם סינון חכם ב `Article.GetInfo()`:

- בדיקת `status_code`

- סינון מילים כמו "captcha", "access denied", "blocked"

- פסילה אם אין `<title>` או `<p>`

תהליך Login – אימות וזיהוי

- **בעיה:** התחברות לא מאובטחת עלולה לחשוף סיסמאות או לאמת משתמשים לא חוקיים.

- **פתרון:**

- סיסמאות נדרשות לעמוד בסטנדרטים של סיסמה חזקה
- סיסמאות נשמרות לאחר הצפנת SHA256 בצד הלקוח.
- שמות משתמש ואימיילים מועברים לשרת בצורה מוצפנת עם RSA.
- אימות דו שלבי: שליחת קוד אימות למייל שמזוהה עם המשתמש.

התקפת Man In The Middle

- **בעיה:** מאזין עלול ליירט מידע רגיש בין הלקוח לשרת.

- **פתרון:**

- הצפנת RSA אסימטרית – גם בקשות וגם תשובות מוצפנות במפתחות ציבוריים.
- חילוק תוכן הכתבות לשניים והצפנת כל חצי בצופן קיסר עם מפתח אחר.
- אין שליחה של מידע קריטי בטקסט פתוח.

התקפת DOS / DDOS

- בעיה: לקוחות רבים עלולים להעמיס על השרת או להפעיל התקפות.
- פתרון: הגבלת הלקוחות ל10 במקביל ופינוי מקום לאחר שלקוחות מתנתקים.

קלטים בעייתיים

- כל קלט מהמשתמש (שם משתמש, סיסמה, אימייל) עובר:

- בדיקות תווים אסורים.
- סינון מילות גנאי.
- אימות פורמט אימייל.

שכבת התעבורה (Transport)

פרוטוקול TCP – לחיצת יד משולשת

- הפרויקט מבוסס על TCP:
- תומך ב *3 Way Handshake* (*SYN, SYN – ACK, ACK*)
- מבטיח אמינות ושליחת הודעות לפי הסדר.

הצפנה:

RSA

RSA הוא אלגוריתם להצפנה אסימטרית, כלומר, משתמשים בשני מפתחות נפרדים:

1. המפתח הציבורי (e, n)

2. המפתח הפרטי (d, n)

כדי להצפין הודעה, המשתמש עושה:

$$c = m^e \bmod n$$

כדי לפענח את ההודעה, הצד שמחזיק את המפתח הפרטי מפענח כך:

$$m = c^d \bmod n$$

SHA-256

SHA-256 היא פונקציית גיבוב קריפטוגרפית אשר פותחה על ידי הסוכנות לביטחון לאומי של ארצות הברית ומשמשת להצפנה חד צדדית (בלתי ניתנת להפיכה) של מידע. פונקציה זו מקבלת קלט באורך כלשהו ומחזירה פלט קבוע באורך של 256 סיביות (32 בתים), בצורה של מחרוזת הקסדצימלית. תהליך הגיבוב הוא חד-כיווני, כלומר, לא ניתן לשחזר את הקלט מהפלט – והוא מתבצע דרך סדרה של פעולות מתמטיות ובינאריות מורכבות, הכוללות חלוקה לבלוקים, הוספת ריפודים (padding), סבבים של חישובים לוגיים, ושימוש בקבועים מספריים קבועים מראש. יתרונה המרכזי הוא בקושי הרב למצוא שני קלטים שונים שיגרמו לפלט זהה.

הצפנת צופן קיסר מותאם אישית:

במהלך פיתוח המערכת, בחרתי להוסיף שכבת אבטחה נוספת לתוכן הכתובות עצמן עקב כך שאורך התוכן גדול והצפנת RSA הייתה איטית מדי. לצורך כך יצרתי מנגנון הצפנה סימטרית, שמתבסס על הצפנת צופן קיסר. הרעיון הוא שלכל כתבה משויכים שני מספרים אקראיים (בין 1 ל-25) המשמים כשני הקודים של הכתבה על מנת להצפינה. בהצפנתי, לקחתי את התוכן, חילקתי אותו לשניים ועבור כל חצי, החלפתי כל אות, באות שנמצאת מספר אותיות אחריה כמספר הקוד ששייך לחצי זה. השרת מצפין את מספרי הקודים בעמצאות RSA ושולח אותם לשרת, מה שמאפשר לפענח את התוכן על ידי הזזה הפוכה צד הלקוח. היתרון במנגנון זה הוא שאני יודע בדיוק איך הוא פועל ובכך אני יכול להשתמש בו ולשנות אותו כרצוני בקוד.

הפעלת המערכת – חולשות כלליות

חולשות אפשריות:

חולשה	טיפול / מניעה
SQL Injection	המנעות משילוב משתנים בסטרינג הפעולות
קלט פוגעני	מסנן מחרוזות אסורות ומילים פוגעניות
כתובות בעייתיות	אלגוריתם סינון בכתבות
התחברות לא חוקית	אימות דו שלבי + הצפנת נתונים
DOS/DDOS	הגבלת כמות חיבורים לשרת
MITM	RSA להצפנה ידנית של כל קלט רגיש

מימוש הפרויקט

חלק א' – סקירת מחלקות ומודולים

מודולים / מחלקות מיובאות

שם המודול	שימוש
socket	ניהול תקשורת TCP בין לקוח לשרת
threading	הפעלת מספר לקוחות בו-זמנית בשרת באמצעות תהליכים
multiprocessing	עיבוד כתבות במקביל (מאפשר עבודה מהירה מול מספר כתובות URL)
pickle	המרת אובייקטים (כמו Article) לבינארי ושליחתם ברשת
smtplib	שליחת אימייל לאימות דו-שלבי
requests	שליפת תוכן מכתבות באינטרנט
bs4	ניתוח HTML מתוך כתבות אינטרנט (BeautifulSoup)
customtkinter	בניית ממשק גרפי (GUI) מודרני
PIL	טעינת אייקונים ותמונות ל-GUI
sqlite3	ניהול מסד נתונים של המשתמשים

hashlib	הצפנת סיסמאות באמצעות SHA256
flask	יצירת שרת מקומי להצגת כתבות בדפדפן
os	פתיחת קובץ הטקסט שמכיל את תוכן הכתבות לאחר יצירתו

מחלקות שפותחו בפרויקט

מחלקה: Article

- **תפקיד:** ייצוג כתבה מהרשת, עם אפשרות להצפנה ושמירת מפתחות ההצפנה.

תכונות:

url - תובת הכתבה

title - כותרת הכתבה (מוצפנת)

content - תוכן הכתבה (מוצפן)

query - החיפוש שהביא את הכתבה

code - מספר להצפנת חצי מהתוכן

code2 - מספר להצפנת החצי השני של התוכן

פעולות:

טענת יציאה	טענת כניסה	פעולה
מאחזרת את התוכן מהאינטרנט, מסן, מצפינה, מחזירה True/False	מספר מונה לכתבות	GetInfo(i)
מחזירה רשימת תווים מוצפנים	מפתח ציבורי וטקסט	RSAencrypt(pk, plaintext)

מחלקה: DataBase

- **תפקיד:** ניהול מסד הנתונים credentials.db, הוספה, אימות ובדיקת משתמשים.

תכונות:

conn - אובייקט החיבור למסד הנתונים

cursor - מצביע לביצוע פעולות

פעולות:

פעולה	טענת כניסה	טענת יציאה
create_table()	אין	יוצרת טבלה חדשה אם הטבלה לא קיימת כבר
CheckIfCanAdd(username, email)	שם ואימייל	בודק אם ניתן להוסיף משתמש (לא קיים כבר)
AddUser(username, password, email)	שם, סיסמה, אימייל	מוסיף את המשתמש לטבלה. מחזיר True/False
CheckIfExists(username, password)	שם וסיסמה	בודק אם קיים משתמש כזה
GetEmailByUsername(username)	שם משתמש	מחזיר את כתובת המייל של המשתמש

מחלקה: Communication

- **תפקיד:** ניהול תקשורת בין הלקוח לשרת והפעלת שרת Flask לצפייה בכתבות.

תכונות:

host - שמירת הִקו של השרת

port - שמירת הפורט עליו רץ השרת

client - חיבור TCP לשרת

flask_process - תהליך נפרד של שרת Flask להצגת כתבות

isOk - מצב החיבור לשרת (True/False)

פעולות:

פעולה	טענת כניסה	טענת יציאה
SendData(message)	מחרוזת הודעה (login/signup)	שולח לשרת, מחזיר תשובת התחברות
GetUrls(query, start, titles)	מחרוזת חיפוש, אינדקס התחלה, כותרות קיימות	מחזיר רשימת כתבות חדשות
OpenInBrowser(url)	כתובת URL	מפעיל Flask ומציג את הכתבה בדפדפן לפי מידע שמקבל מהשרת

מחלקה: Login

- תפקיד: GUI להתחברות והרשמה עם אימות דו-שלבי.

תכונות:

server - תקשורת עם השרת

login - החלון הראשי עליו רץ חלון ה-root

error - לייבל שמתאר תקלה בתהליך ההתחברות

signup - החלון עליו רץ חלון ה-top

on_success - פקודה המקשרת בחזרה לתכנית הראשית לאחר התחברות למערכת

username, passwordE, email - CTKEntry אימיל, סיסמה, לשם

פעולות:

פעולה	טענת כניסה	טענת יציאה
toggle_password()	אין	משנה את מצב הסיסמה בין כוכבית לאותיות.
DisplayInstructions()	אין	פותח את החלון המכיל את הדרישות לסיסמה
check_password(password)	סיסמה	בודקת את הסיסמה אם היא עומדת בדרישות
VerifyMail(mail)	כתובת מייל	פותחת את מסך האימות

Check()	אין	בודק האם הקוד שהוכנס תואם את הקוד שנשלח
RSAdecrypt(pk, text)	מפתח פרטי וטקסט	מפענחת מחרוזת שמוצפנת ב-RSA
()login	אין	מציג את מסך ההתחברות
()signup	אין	מציג את מסך ההרשמה
checkifcancreate()	אין	שואל את השרת אם אפשר ליצור משתמש ויוצר אותו אם אפשר, ואם לא מציג שגיאה
checkifexists()	אין	שואל את השרת אם קיים משתמש, במידה וכן, מבצע אימות קוד שנשלח במייל, במידה ולא מציג שגיאה

מחלקות: RegularQuery, UriQuery

- **תפקיד:** בניית פקודת חיפוש לשרת לפי סוג הבקשה (רגיל או URL).
- **יירושה:** יורשות את המחלק QueryType

תכונות:

query - מחרוזת חיפוש

start - מספר כתבות שכבר קיימות אצל הלקוח

פעולות:

פעולה	טענת כניסה	טענת יציאה
setQuery(query)	מחרוזת חיפוש	מגדירה את הפקודה להצפנה ושימוש
format_for_sending()	אין	מחזירה מחרוזת הפקודה (.../uRI/search)

מחלקה: Save

- **תפקיד:** שמירת תוכן הכתבות במחשב בקובץ טקסט.

תכונות:

title - כותרת הכתבה

content - תוכן הכתבה

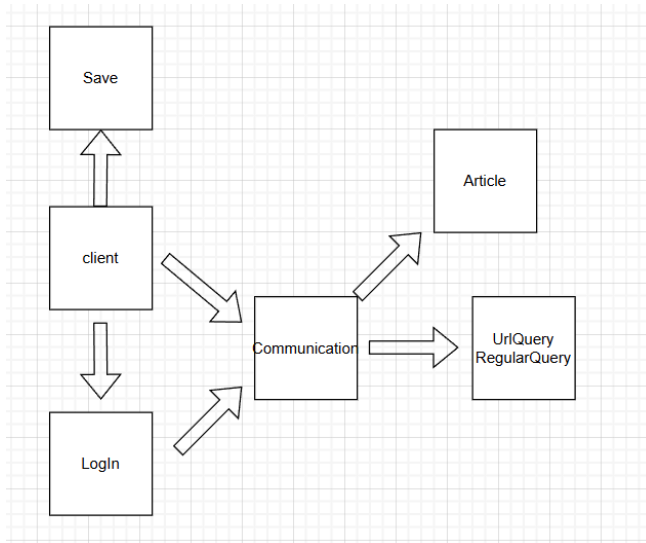
url - הקישור לכתבה

פעולות:

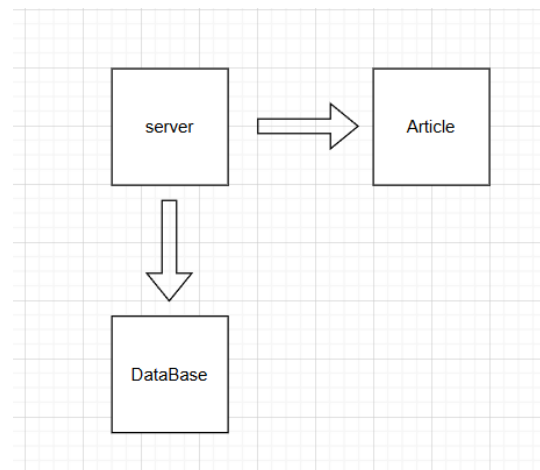
פעולה	טענת כניסה	טענת יציאה
cleanTitle()	אין	מנקה את כותרת הכתבה מאותיות שס אוסר מלשים בכותרת קובץ
saveToComputer()	אין	שומרת את הכתבה במחשב בקובץ טקסט

קשרי הכלה/ירישה

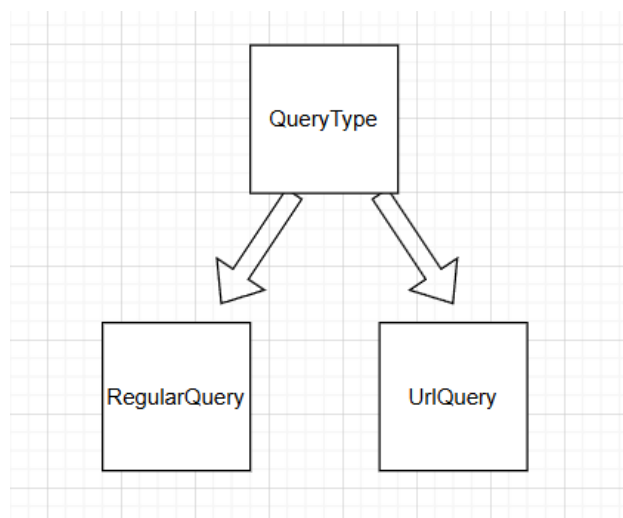
הכלה בצד הלקוח:



הכלה בצד השרת:



ירישה בפרויקט:



חלק ב'

קטעי קוד רלוונטיים והסברים

בעיה אלגוריתמית 1: סינון כתבות שאינן ניתנות לקריאה

מטרה:

למנוע שליחה של כתבות שהן עמודים ריקים, חסומים, או עם CAPTCHA.

קטע קוד מתוך `Article.GetInfo()`:

```
try:
    response = requests.get(self.url, headers=headers, timeout=10)
except requests.RequestException as e:
    print(f"error in article: Request failed for {self.url}: {e}")
    return False

soup = BeautifulSoup(response.content, 'html.parser')

title_tag = soup.find("title")
self.title = ""
if title_tag:
    self.title = title_tag.get_text(strip=True)

content_parts = []

for heading in soup.find_all(['h1', 'h2', 'h3', 'h4', 'h5', 'h6']):
    content_parts.append(heading.get_text(strip=True)+"\n")

for paragraph in soup.find_all('p'):
    content_parts.append(paragraph.get_text(strip=True)+"\n")

self.content = ' '.join(content_parts).strip()

if content_parts == []:
    self.content = "this article doesn't contain any text information, try to open it in browser"

if response.status_code != 200 or not self.title:
    return False

blocked_titles = [
    "error", "403", "no title found for this article", "homepage",
    "just a moment", "captcha", "blocked"
]

blocked_content_words = [
    "access denied", "blocked", "please verify", "security check",
    "captcha", "robot", "user-agent"
]
```

```
title_lower = self.title.lower()
content_lower = self.content.lower()

for word in blocked_titles:
    if word in title_lower and word not in self.query.lower():
        return False

for word in blocked_content_words:
    if word in content_lower:
        return False
return True
```

הסבר:

- כל כתבה נבדקת: האם הכילה תוכן ממשי? האם היא חסומה? האם יש מילים חשודות?
- נשללות כתבות לא שמישות כבר בשרת עוד לפני שהן עוברות ללקוח ובכך חוסכות זמן מיותר בהעברה.

בעיה אלגוריתמית 2: שליחת תוכן ארוך בצורה אמינה

מטרה:

לשלוח מאמרים שלמים ב־socket בלי שהנתונים ייחתכו.

קטע קוד – בצד השרת:

```
articlesInBytes = pickle.dumps(articles)

data_length = len(articlesInBytes)

client_socket.sendall(data_length.to_bytes(8, 'big'))

client_socket.sendall(articlesInBytes)
```

קטע קוד – בצד הלקוח:

```
data_length_bytes = self.client.recv(8)

data_length = int.from_bytes(data_length_bytes, 'big')

received_data = b""

while len(received_data) < data_length:

    msg = self.client.recv(4096)

    if not msg:

        break

    received_data += msg

articles = pickle.loads(received_data)
```

הסבר:

- נשלח קודם אורך המידע (8 בתים) כדי שהלקוח ידע בדיוק כמה לקרוא.
- קריאה בלולאה מבטיחה שהמידע נקלט במלואו.
- שימוש ב-pickle מאפשר שליחת אובייקטים מורכבים כמו כתבות.

קטע קוד יפה: הצפנה סימטרית מותאמת אישית לכתבות

מטרה:

לשמור על תוכן הכתבות בצורה מוצפנת ייחודית, עם שני קודים סודיים משתנים לכל כתבה.

קטע קוד – מתוך encrypt() בשרת:

```
def encrypt(text, code1, code2):
    encrypted = ""
    for i in range(2):
        if i == 0:
            code = code1
            half = text[:len(text) // 2]
        else:
            code = code2
            half = text[len(text) // 2:]

        result = ''
        for char in half:
            if 'a' <= char <= 'z':
                result += chr((ord(char) - ord('a') + code) % 26 + ord('a'))
            elif 'A' <= char <= 'Z':
                result += chr((ord(char) - ord('A') + code) % 26 + ord('A'))
            else:
                result += char
        encrypted += result

    return encrypted
```

הסבר:

- הצפנה בסגנון צופן קיסר מיוחדת, הטקסט מחולק לחצי ולכל חצי יש קוד שונה וכל האותיות בחצי מוזזות לפי הקוד של החצי.
- שיטה פשוטה אך יעילה להסתרת התוכן עד לפענוח בצד הלקוח.

חלק ג':

בדיקות שתוכננו מראש (בשלב האפיון)

מטרת הבדיקה	מה בוצע בפועל	תוצאת הבדיקה	טיפול בתקלות
לבדוק שהרשמה עם פרטי משתמש חוקיים עובדת	הזנה של שם משתמש "barak", אימייל תקני וסיסמה תקפה	המערכת קיבלה את הבקשה ושלחה "True"	לא נדרשה פעולה נוספת
לבדוק סינון מילים אסורות בשם משתמש	הזנה של שם כמו "racistguy123"	המערכת זיהתה מילה אסורה ושלחה "forbidden words"	לא נדרשה פעולה נוספת
לבדוק אימייל לא חוקי	ניסיון להירשם עם אימייל: "@gmail.com"	המערכת שלחה "email problem"	לא נדרשה פעולה נוספת
לבדוק התחברות תקינה	הזנת שם וסיסמה של משתמש קיים	קוד אימות נשלח למייל והתקבלה תשובת "True/code/mail"	לא נדרשה פעולה נוספת
לבדוק התחברות שגויה	הזנת סיסמה שגויה	המערכת שלחה "False"	לא נדרשה פעולה נוספת
לבדוק חיפוש רגיל עם מונח חדשותי	חיפוש "israel"	השרת החזיר כתבות, הוצגו ב-GUI אך לא בצורה אחידה	שיניתי את הדרך בה הקוד מציג את הכתבות.

לבדוק חיפוש לפי URL	חיפוש לפי query="https://www.baltimoreravens.com/news/derrick-henry-ravens-two-year-extension-contract-2025..."	הוצגה כתבה אחת מתוך הכתובת	לא נדרשה פעולה נוספת
לבדוק קלט ריק	הזנת חיפוש ריק	הבקשה נשלחה לשרת והחזירה כתבות כלליות	נוספה בדיקת if query == "" וחסימה אם מתקיים בלקוח
לבדוק הצפנה/פענוח תקינים	שליחת כתבה מוצפנת ופענוח בלקוח	התוכן הוצג בשלמותו לאחר פענוח	לא נדרשה פעולה נוספת
לודא שמאמרים חסומים לא עוברים	ניסוי עם אתר מוגן (captcha / 403)	רוב הכתבות נחסמו אך כמה הצליחו לעבור	הוספתי עוד מילים לרשימת המילים לבדיקה ב Article.GetInfo()

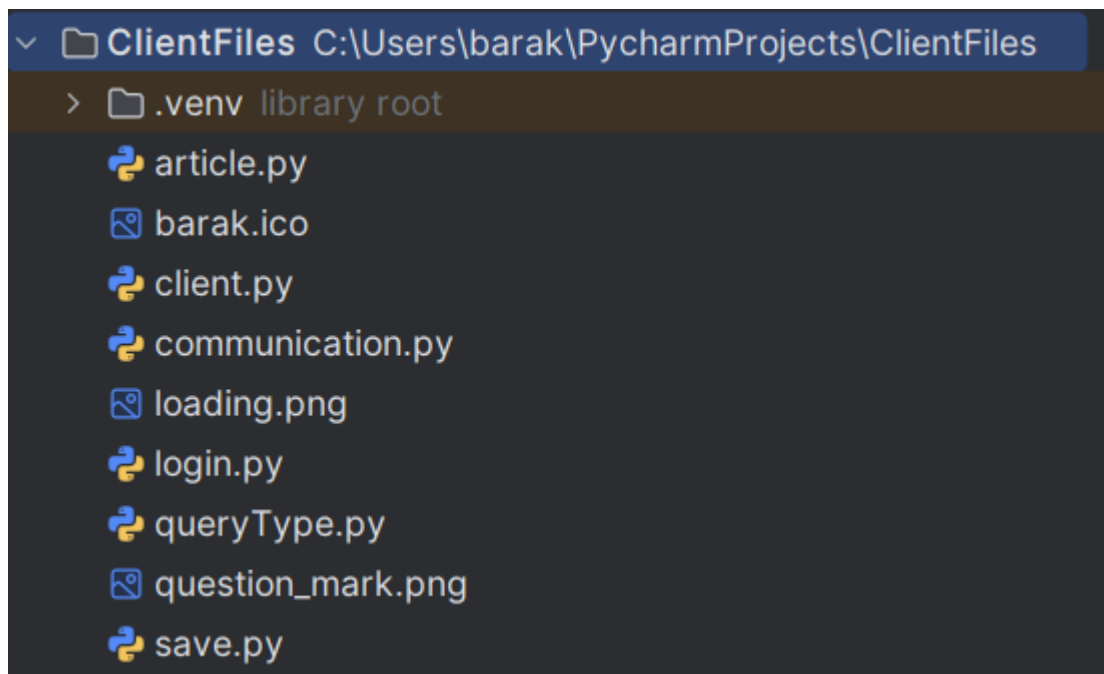
בדיקות נוספות שבוצעו במהלך הפיתוח

מטרת הבדיקה	מה בוצע בפועל	תוצאת הבדיקה	כיצד נפתרו בעיות
לבדוק התמודדות עם כתבות ארוכות מאוד	חיפוש באינטרנט בשביל כתבות ארוכות ואז חיפושן במערכת	התוכן התקבל חלקית	נוסף מנגנון שליחת אורך <code>to_bytes(8, 'big')</code> לפני המידע
לבדוק זמני תגובה תחת עומס	חיבור 10 לקוחות במקביל	הלקוח ה-11 נדחה עם <code>"server_full"</code>	נוספה בדיקה בשרת לפני קבלת חיבור חדש
לבדוק תקינות Flask להצגת כתבות	פתיחת כתבה בדפדפן מקומי דרך כפתור GUI	התוכן הוצג כראוי ב- <code>localhost:5000</code>	לא נדרשה פעולה נוספת
לבדוק ניתוק לא תקין של לקוח	סגירה פתאומית של לקוח בזמן התחברות	השרת זיהה ניתוק, אך אינו הסיר את החיבור מהרשימה	נוספה פונקציה <code>RemoveClient</code> שסוגרת את החיבור עם קליינט מסויים ומוציאה אותו מהרשימה

מדריך למשתמש

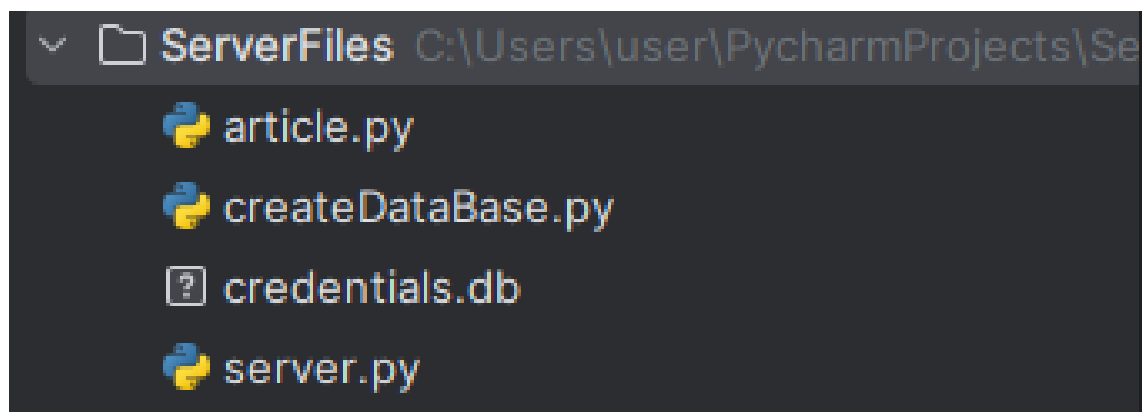
פירוט כלל קבצי המערכת – עץ קבצים

קבצי לקוח:



קבצי שרת:

=



התקנת המערכת

הסביבה הנדרשת

פירוט	דרישה
מומלץ להשתמש ב-Windows 10 ומעלה לצורך תאימות מלאה לספריות.	מערכת הפעלה
מומלץ להשתמש בגרסה 3.9 ומעלה לצורך תאימות מלאה לספריות.	Python
חובה לצורך התחברות לשרת, חיפוש כתבות ואימות מיילים.	חיבור לאינטרנט

הכלים הדרושים

חבילה / ספרייה	תפקיד	התקנה
requests	שליפת תוכן מהאינטרנט	pip install requests
bs4	ניתוח HTML של כתבות	pip install beautifulsoup4
customtkinter	GUI מודרני	pip install customtkinter
smtplib	שליחת מיילים	לא נדרש
flask	הצגת כתבות בדפדפן	pip install flask
duckduckgo_search	חיפוש כתבות	pip install duckduckgo-search
google search	חיפוש כתבות	pip install googlesearch-python
PIL	טעינת תמונות	pip install pillow
os	פתיחת קבצים	לא נדרש

מיקומי קבצים

- כל קבצי השרת (כולל מסד הנתונים) בפרויקט משלהם
- כל קבצי הלקוח (כולל תמונות ואייקונים) בפרויקט משלהם

נתונים התחלתיים

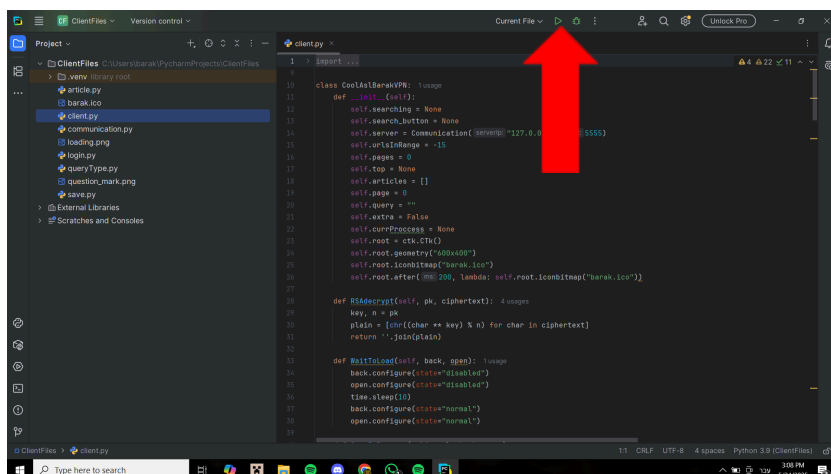
- אין משתמשים קבועים מראש, כל משתמש נרשם דרך המערכת.
- תהליך ההרשמה דורש:
 - אימייל תקני
 - שם תקין ללא מילים לא נאותות
 - סיסמה תקינה (+8 תווים, אותיות גדולות, ספרות ועוד)
 - אימות קוד הנשלח בדוא"ל

רשת

- נדרש שינוי הכתובת שבה client.py רץ, לפי ה IP של המחשב שממנו מריצים את השרת.
- בעת הצגת כתבות בדפדפן, נפתח שרת Flask זמני ב-`http://127.0.0.1:5000`
- נדרשת פתיחת פורטים אלו אם יש חומת אש.

אופן הפעלת המערכת:

מריץ את `client.py`



התחברות למערכת

על הלקוח להכניס את פרטי המשתמש שלו כמו שניתן לראות

בדוגמה הזאת:

אם רוצים לראות את הסיסמה במקום שתהיה מוצגת כ " * " "

ניתן ללחוץ על כפתור ה"show".

במידה ואין ללקוח משתמש, עליו ללחוץ על כפתור

ה"sign up here" על מנת ליצור משתמש.

לאחר שהלקוח הכניס את פרטיו עליו ללחוץ על"log in",

על מנת להתחבר למערכת.

אם פרטיו לא תואמים משתמש שקיים כבר במערכת תוצג לו הערה "Account doesn't exist" בצורה זו:

אם פרטיו כן תואמים משתמש שקיים במערכת, יפתח לו מסך חדש

ובו עליו להכניס את הקוד שנשלח לו לכתובת המייל איתה נרשם

למערכת:

במידה והכניס קוד שגוי תופיע לו הודעה:

במידה והכניס קוד נכון הוא יתחבר למערכת ויועבר למסך הראשי (הסבר עליו בהמשך):

*דוגמה לאיך נראית הודעה מהמערכת



הרשמה למערכת

A registration form for BarakVPN. It has a title 'Welcome to BARAK VPN' and a subtitle 'sign up for the best project in the world here'. There are three input fields: 'enter your email' with a placeholder 'enter email here', 'enter your username' with a placeholder 'enter username here', and 'enter your password' with a placeholder 'enter password here' and a 'show' button. Below the fields is a 'SIGN UP' button and a link 'log in here' for users who already have an account.

על הלקוח להכניס את הפרטים איתם רוצה להירשם למערכת, לפי הסדר שכתוב במסך:

על הלקוח לוודא שהאימייל שלו תקני, כלומר, אינו מכיל אותיות גדולות, משתמש בדומיין אמיתי, ומשתמש רק באותיות מקובלות.

על הלקוח לוודא ששם המשתמש שלו לא מכיל מילים פוגעניות.

על הלקוח לוודא שהסיסמה שלו עומדת בתנאי החוזקה, את תנאי החוזקה ניתן לראות כאשר לוחצים על כפתור הסימן שאלה משמאל לשדה שבו מכניסים את הסיסמה.

A dialog box titled 'Password Instructions' with the heading 'Passwords must:'. It lists the following requirements: 'contain 8 or more letters', 'contain at least one upper case letter', 'contain at least one lower case letter', 'contain at least one number', 'cant have a series of at least 3 following numbers', and 'contain the same character 3 or more times in a row'.

כאשר לוחצים עליו נפתח מסך זה:

לאחר שהלקוח ממלא את הפרטים ומוודא שהם עומדים בתנאים, על הלקוח ללחוץ על כפתור ה"SIGN UP" על מנת להרשם למערכת, במידה והפרטים שלו עומדים בתנאים ואינם קיימים כבר במערכת, המשתמש ירשם למערכת בהצלחה ויעבור למסך הראשי.

במידה והפרטים שלו לא עומדים בתנאים או כבר קיימים יוצגו לו הערות אלו בהתאם:

שם משתמש לא תקין

The screenshot shows the 'Welcome to BARAK VPN' sign-up page. The email field contains 'barak@gmail.com', the username field contains 'racistguy123', and the password field contains 'BarakVPN1'. A red error message 'please use prettier words' is displayed below the password field. The 'SIGN UP' button is visible, along with a 'log in here' link for existing users.

כתובת אימייל לא תקינה

The screenshot shows the 'Welcome to BARAK VPN' sign-up page. The email field contains 'Barak@gmail.com', the username field contains 'barakVPN', and the password field contains 'BarakVPN1'. A red error message 'invalid email address' is displayed below the password field. The 'SIGN UP' button is visible, along with a 'log in here' link for existing users.

משתמש קיים כבר

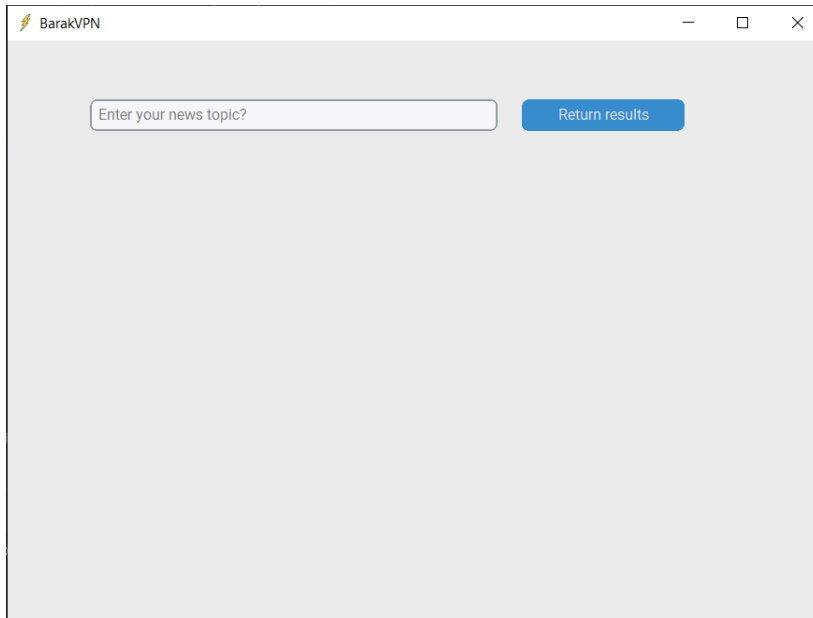
The screenshot shows the 'Welcome to BARAK VPN' sign-up page. The email field contains 'barak@gmail.com', the username field contains 'barak', and the password field contains 'BarakVPN1'. A red error message 'User already exists' is displayed below the password field. The 'SIGN UP' button is visible, along with a 'log in here' link for existing users.

סיסמה לא תקינה

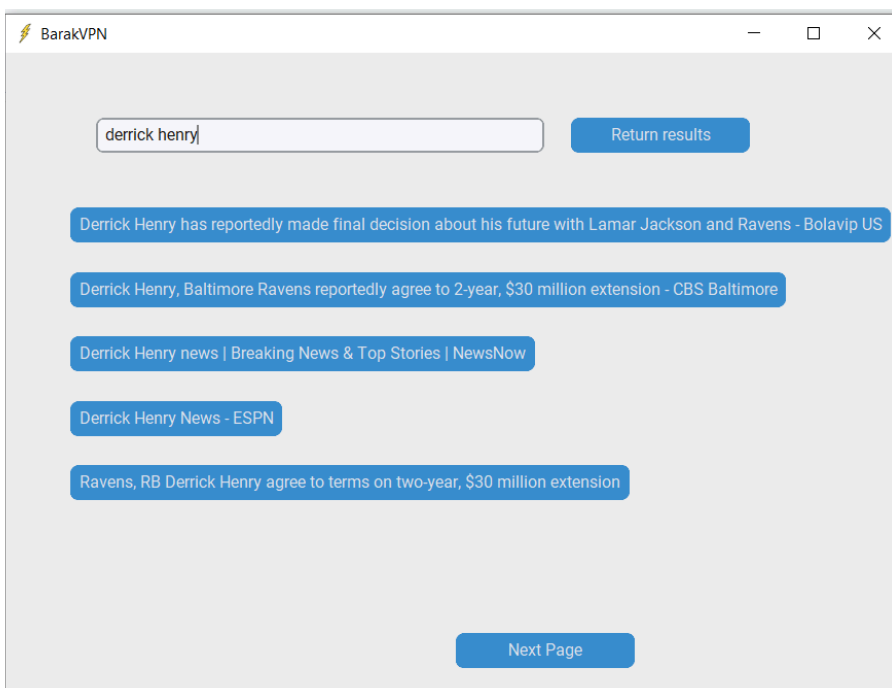
The screenshot shows the 'Welcome to BARAK VPN' sign-up page. The email field contains 'barakVPN@gmail.com', the username field contains 'barakVPN', and the password field contains '123'. A red error message 'please follow the password manual' is displayed below the password field. The 'SIGN UP' button is visible, along with a 'log in here' link for existing users.

חיפוש כתבות לפי נושא או URL וקריאת תוכן הטקסט שלהם

לאחר שהלקוח סיים את ההתחברות/הרשמה למערכת הוא מועבר למסך הראשי בו הוא יכול לבצע חיפוש לפי מילות מפתח או קישורים ספציפיים שהוא מעוניין בהם.



על הלקוח להקליד את מה שברצונו לחפש בשורת החיפוש וכשהוא מסיים, ללחוץ על כפתור ה"Return Results"

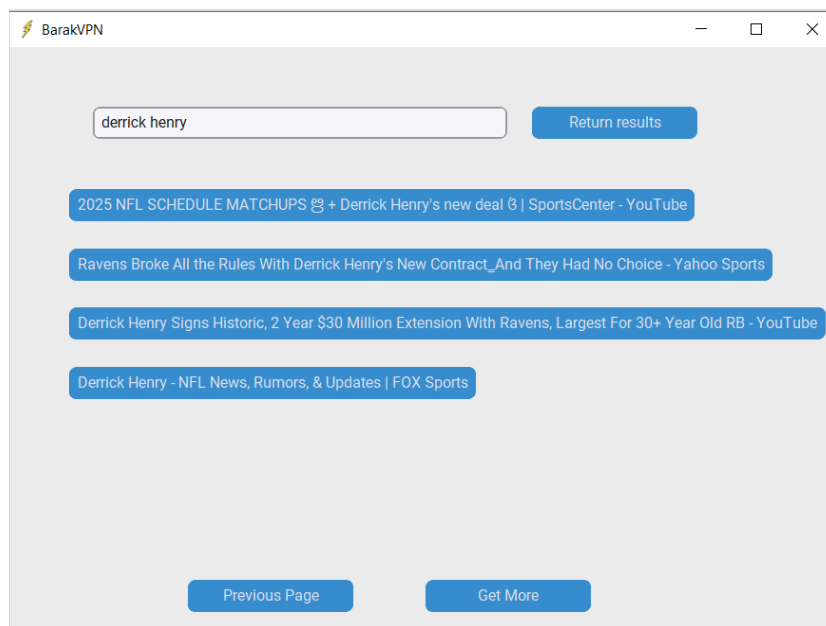


כך יוצגו תוצאות החיפוש למשתמש:

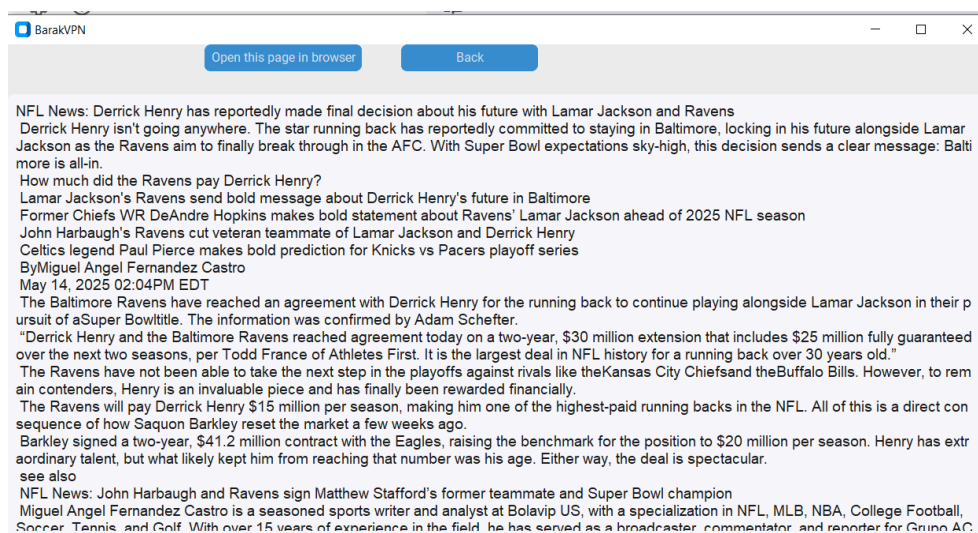
התוצאות מפוצלות לדפים, בכל דף יש 5 כתבות או פחות אם זה העמוד האחרון, על מנת לעבור עמוד יש ללחוץ על כפתור ה"Next Page".

אם הלקוח רוצה לחפש משהו אחר עליו לשנות את מה שכתוב בשורת החיפוש וללחוץ על כפתור ה"Return Results" שוב.

במידה והלקוח לא מרוצה מהכתבות שקיבל רוצה עוד עליו ללחוץ על כפתור ה"Get More" אשר מופיע כאשר הוא בדף האחרון של התוצאות:



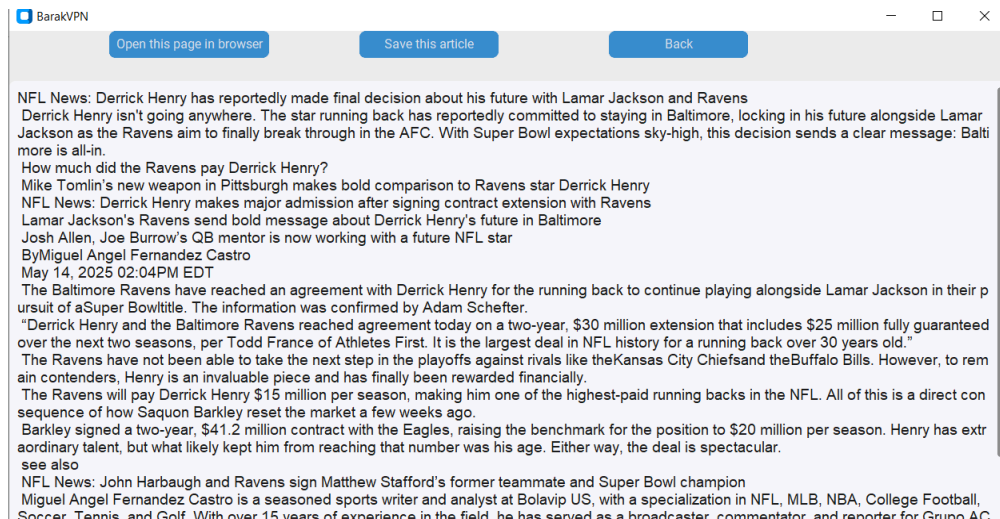
אם הלקוח מתעניין באחת מהכתבות הוא יכול ללחוץ על הכותרת שלה והוא מועבר למסך הזה שבו הוא יכול לקרוא את תוכן הטקסט של הכתבה:



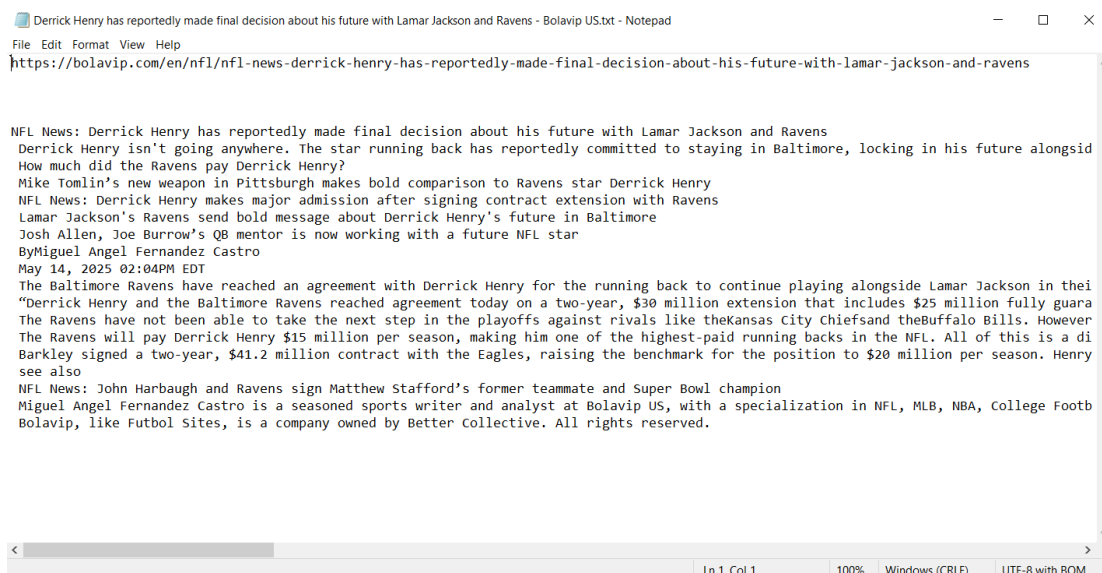
במידה והלקוח רוצה לחזור שוב למסך הכתבות עליו ללחוץ על כפתור ה"Back"

פתיחת כתבות בשרת פרטי באינטרנט ושמירת הכתבות במחשב

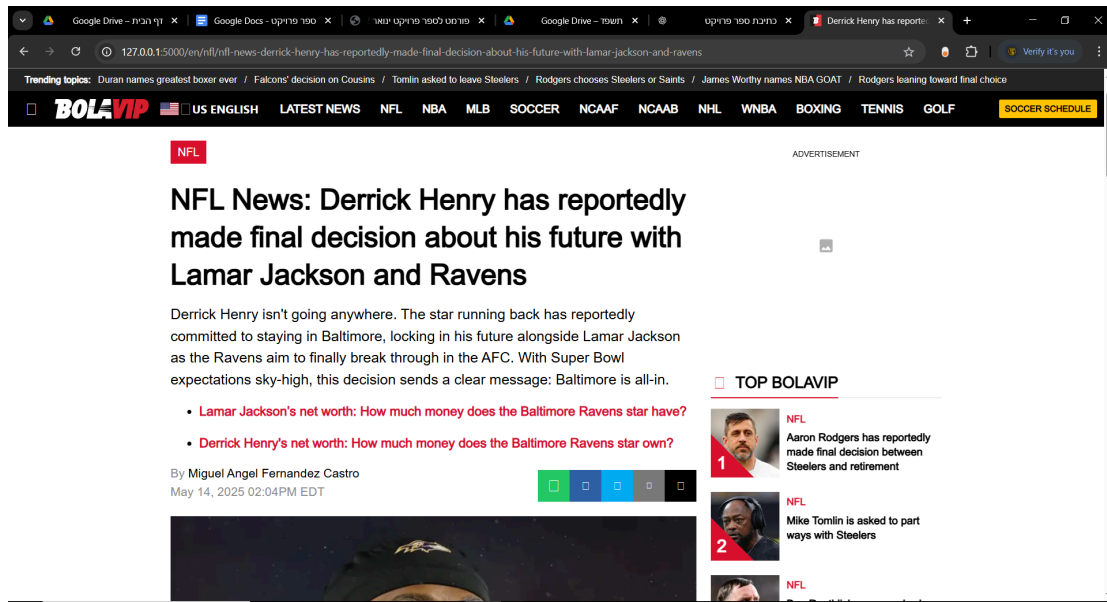
במסך קריאת תוכן הטקסט:



ללקוח יש אפשרות לשמור את הכתבה בקובץ טקסט בלחיצה על הכפתור "Save this article", כאשר הוא לוחץ על כפתור זה הכתבה תשמר במחשב ויפתח הקובץ שבה הוא שמור. דוגמה:



ללקוח יש גם אפשרות ללחוץ על כפתור ה "Open this page in browser", כאשר הוא לוחץ על כפתור זה, יפתח שרת אינטרנט פרטי שמציג את התוכן של האתר בצורה המקורית שלו:



סיכום אישי / רפלקציה

עבודתי על פרויקט BarakVPN נמשכה חודשים רבים וכללה תכנון, כתיבה, ניסוי וטעייה, ועיבוד חוזר של רעיונות עד שהגעתי לפתרון עובד ומרצה. לאורך הדרך היו גם הצלחות וגם אתגרים, אך כל שלב תרם רבות ליכולות הלמידה והפיתוח שלי.

אחת ההצלחות המשמעותיות הייתה יצירת מערכת Client-Server שלמה, מהקלט של המשתמש, דרך התקשורת המוצפנת, ועד לשליפת תכנים מהאינטרנט והצגתם בצורה נקייה ונגישה.

נתקלתי בקשיים רבים בעיקר עם החיפוש ברשת ושליחת הכתבות שעשה לי הרבה בעיות עם קבלת התוכן, הוא היה נקטע באמצע או מגיע יותר מדי גם מהשליחות הבאות, ישבתי שעות רבות מהימים שלי עד שהצלחתי למצוא פיתרון שפועל ולגרום לו לפעול ביחד עם מערכת ההצפנה שלי.

אך בכל פעם שהצלחתי להתגבר על קושי וראיתי את הדבר שעבדתי עליו במשך שעות ארוכות רבות פועל, הרגשתי תחושת גאווה ושמחה עצומה. העבודה על הפרויקט לימדה אותי להתגבר על קשיים, לחשוב בדרכים יצירתיות, לעבוד גם כשאני לא רוצה ולא כיף לי, ובכללי פיתחה את יכולות העבודה שלי והמשמעת שלי, מה שיעזור בדברים רבים אחרים בחיים שלי שהם לא הפרויקט.

לאורך העבודה על הפרויקט נחשפתי לתכנים שלא הכרתי לפני וגם העמקתי את הידע והשליטה שלי בתכנים שכן ידעתי, לקחתי את התוכנות שכתבתי כחלק משיעורי בית הספר כמו למשל אפליקציית ה customtkinter הבסיסית והשרת מרובה משתמשים, ובניתי עליהם, בכל פעם הוספתי דברים אחרים שהייתי צריך לפרויקט, בין אם הם היו פשוט שימושים אחרים של מה שכבר למדתי או דברים שלמדתי לגמרי בעצמי, כמו החיפוש ברשת, שימוש בbs4 כדי לחלץ מידע ספציפי מתוכן הHTML או שימוש בpickle בשביל לשלוח עצמים ארוכים.

בכל פעם הוספתי משהו, ומדי פעם מחקתי דברים שהפריעו, אך בסופו של דבר יצרתי מערכת שפועלת לגמרי, שלוקחת את הדברים שלמדנו בבית ספר במהלך השנים, משרדגת אותם לרמות יותר גבוהות ומוסיפה עוד הרבה תכנים שלמדתי בעצמי.

העבודה על הפרויקט נתנה לי כלים רבים שאקח איתי לחיים, גם כלים שקשורים לתכנות וגם כלים לחיים, במהלך העבודה על הפרויקט ליטשתי את צורת הכתיבה שלי, למדתי מוסכמות בעולם התכנות, גיליתי אתרים שימושיים ללמידה, שיפרתי את החשיבה האלגוריתמית שלי ושכללתי את הקישורים שלי בתור מתכנת בכללי.

אך מעבר ליכולות התכנות שרכשתי מהפרויקט רכשתי גם יכולות מנטליות ומחשבניות חשובות מאוד שיעזרו לי בתחומים רבים אחרים מחוץ לעולם התכנות, כמו המשמעת לשבת מול המחשב ולעבוד גם אחרי יום ארוך, גם היכולת לשבת במשך שעות רבות ולעבוד על אותה הבעיה עד שלבסוף אני מצליח לפתור אותך והכי חשוב, הפרויקט עזר לי לפתח חשיבה יצירתית ויכולת פתירת בעיות שאני מאמין שתהיה שימושית מאוד בעתיד.

במהלך הפרויקט נעזרתי הרבה בחברים, בין אם זה בקבלת דעה לגבי נושא מסוים, התייעצות לגבי מה כדאי לי לעשות, או עזרה בבעיות שהיו לי בקוד. shoutout למאמין.

נעזרתי גם במורים שלי בשאלות שהיו לי בפרויקט, בתחומים רבים, למשל ההצפנות, רעיונות על פיצ'רים שאפשר להוסיף כדי לשפר את הפרויקט ולהוסיף אליו עוד תוכן.

דבר שעזר לי מאוד הוא קבלת חוות דעת מהמורים שלי, מהחברים שלי או אפילו מהמשפחה שלי על ממשק המשתמש שלי ועל הדרכים שבהן מיממשי את היכולות של המערכת, פעמים רבות שיניתי לטובה דברים במערכת כתוצאה מכך שמישהו הציע לי דרך אחרת לעשות אותם, מה שעזר לי לשפר את המערכת שלי על ידי שילוב דעות רבות.

בראייה לאחור, אם הייתי חוזר להתחלה הייתי משקיע יותר זמן בתכנון מראש במקום לחשוב על זה בזמן שאני עובד על זה, בנוסף, הייתי מתחיל לעבוד על הפרויקט מוקדם יותר כדי שאני אוכל להוסיף עוד פיצ'רים ואהיה פחות לא לחוץ בזמן.

עוד דבר שיכולתי לשפר הוא ניהול גרסאות, הרבה פעמים מצאתי את עצמי מחפש קוד ישן שעבד, אחרי שכבר שיניתי דברים ולא גיביתי ואז הייתי צריך לשבת ולשכתב את הקוד הישן, מה שלקח הרבה זמן.

אם היו לי משאבים נוספים, הייתי משלב בפרויקט מודל בינה מלאכותית שידע לדרג את הכתבות בצורה אוטומטית, בהתבסס על קריטריונים כמו רלוונטיות, איכות התוכן וזמינות התוכן. בנוסף, הייתי מפתח גרסה מותאמת למכשירים ניידים וגרסה אינטרנטית מלאה, שתאפשר חוויית משתמש נוחה וגמישה בכל פלטפורמה, עם ממשק שמתאים גם למסכים קטנים וגם לדסקטופ.

שאלות חקר שצצו לי

- האם ניתן להקים מערכת VPN אמיתית (לתקשורת) על בסיס אותו עיקרון?
- האם אפשר לבנות מערכת VPN שמתחברת ישירות לאינטרנט בלי צורך באפליקציה?tkinter

אני מודה למורים שלי שעזרו לי בכל השאלות שהיו לי, לחברים שלי שעזרו לי לפתור חילוקי דעות שהיו לי עם עצמי והתלבטויות.

ביבליוגרפיה

אתר PyPi, למידת ספריות החיפוש בגוגל וב"duckduckgo"-

<https://pypi.org/project/duckduckgo-search/>,
<https://pypi.org/project/googlesearch-python/>

אתר w3schools, למידה על ספריית requests -

https://www.w3schools.com/python/ref_requests_response.asp

ערוץ היוטיוב "All about python", למידה על ספריית smtp lib

https://www.youtube.com/watch?v=S465v4mWsRg&ab_channel=AllAboutPython

אתר geeksforgeeks, למידה על ספריית bs4

<https://www.geeksforgeeks.org/beautifulsoup-scraping-paragraphs-from-html/>

אתר python documentation, למידה על ספריית pickle

<https://docs.python.org/3/library/pickle.html>

הקבצים במודל של ליאת סגל, בשביל קודים בסיסיים להכניס למערכת שלי ולבנות עליהם אלגוריתמים יותר מסובכים.

קובץ הראשי של הלקוח - client.py

```

import threading
import time
from communication import Communication
import customtkinter as ctk
import webbrowser
from login import LogIn
import article
from PIL import Image
import save

class CoolAslBarakVPN:
    def __init__(self):
        self.searching = None
        self.search_button = None
        self.server = Communication("127.0.0.1", 5555)
        self.urlsInRange = -15
        self.pages = 0
        self.top = None
        self.articles = []
        self.page = 0
        self.query = ""
        self.extra = False
        self.currProccess = None
        self.root = ctk.CTk()
        self.root.geometry("600x400")
        self.root.iconbitmap("barak.ico")
        self.root.after(200, lambda: self.root.iconbitmap("barak.ico"))

    def RSAdecrypt(self, pk, ciphertext):
        key, n = pk
        plain = [chr((char ** key) % n) for char in ciphertext]
        return ''.join(plain)

    def WaitToLoad(self, back, open):
        back.configure(state="disabled")
        open.configure(state="disabled")
        time.sleep(10)
        back.configure(state="normal")
        open.configure(state="normal")

    def SaveArticle(self, article, save):
        file = save.Save(article.getTitle(), article.getContent(),
        article.getUrl())
        isGood = file.saveToComputer()
        if not isGood:
            ctk.CTkLabel(self.toplevel, text="can't save this article",
            font=("Helvetica", 10), text_color="red").place(x=350, y=30)

```

```

def OpenInBrowser(self, url, back, open):
    self.currProccess = self.server.OpenInBrowser(url)
    webbrowser.open_new_tab("http://127.0.0.1:5000")
    threading.Thread(target= self.WaitToLoad, args=(back,
open)).start()

def displayURLcontent(self, article):
    self.top.withdraw()
    self.toplevel = ctk.CTkToplevel(self.root)
    self.toplevel.title("BarakVPN")
    self.toplevel.geometry("1000x500")
    self.toplevel.iconbitmap("barak.ico")
    self.toplevel.after(200, lambda:
self.top.iconbitmap("barak.ico"))

    textbox = ctk.CTkTextbox(self.toplevel, font=("Helvetica", 15),
height=450, width=1000)
    textbox.place(x=0, y=50)

    textbox.insert("1.0", article.getContent())

    backToSearches = ctk.CTkButton(self.toplevel, text="Back",
command=lambda: self.returnToMainPage(self.toplevel))
    backToSearches.place(x=600, y=0)

    openinbrowser = ctk.CTkButton(self.toplevel, text="Open this
page in browser", command=lambda: self.OpenInBrowser(article.getUrl(),
backToSearches, openinbrowser))
    openinbrowser.place(x=100, y=0)

    save = ctk.CTkButton(self.toplevel, text="Save this article",
command=lambda:
self.SaveArticle(article, save))
    save.place(x=350, y=0)

def returnToMainPage(self, toplevel):
    toplevel.withdraw()
    if self.currProccess and self.currProccess.is_alive():
        self.currProccess.terminate()
    self.top.deiconify()

def searchUrls(self):
    self.search_button.configure(state="disabled")
    self.urlsInRange = len(self.articles)

    titles = [a.getTitle() for a in self.articles]

    photo = Image.open("loading.png")
    ctk_image = ctk.CTkImage(light_image=photo, dark_image=photo,
size=(400, 400))
    photolabel = ctk.CTkLabel(self.top, image=ctk_image, text="")
    photolabel.place(x=130, y=100)

```

```

        self.top.update()
        articles = self.server.GetUrls(self.query, len(self.articles),
titles)

        photolabel.destroy()

        for a in articles:
            a.setTitle(self.RSAdecrypt((5213, 7373),a.getTitle()))
            url = self.RSAdecrypt((5213, 7373), a.getUrl())
            a.setCode(int(self.RSAdecrypt((5213,7373), a.getCode())))
            a.setCode2(int(self.RSAdecrypt((5213,7373), a.getCode2())))
            a.setUrl(url)

            a.setContent(self.server.decrypt(a.getContent(),
a.getCode(), a.getCode2()))

            if not articles and not self.articles:
                ctk.CTkLabel(self.top, text="an error occurred with your
search, please try again later",
                                text_color="red",
font=("Helvetica", 12)).place(x=100, y=90)
                self.searching = False
                self.search_button.configure(state="normal")
            else:
                if not articles:
                    ctk.CTkLabel(self.top, text="couldn't find any more
articles",
                                text_color="red", font=("Helvetica",
12)).place(x=120, y=85)

                self.articles += articles
                if len(self.articles) % 5 == 0:
                    if self.extra:
                        self.pages = len(self.articles)// - 1
                    else:
                        self.pages = len(self.articles) // 5
                else:
                    if self.extra:
                        self.pages = len(self.articles)//5
                    else:
                        self.pages = len(self.articles) // 5 + 1

                self.DisplayUrls()

def ShowNextPage(self):
    self.page += 1
    self.DisplayUrls()

def ShowPreviousPage(self):
    self.page -= 1
    self.DisplayUrls()

def DisplayUrls(self):

```



```

for widget in self.top.winfo_children():
    if widget.winfo_y() >= 120:
        widget.destroy()

if self.page == self.pages-1:
    startnum = len(self.articles) - len(self.articles)%5 - 1
    numarticles = len(self.articles) % 5
    if numarticles == 0:
        startnum = startnum - 5
        numarticles = 5

    else:
        startnum = (self.page)*5-1
        numarticles = 5

    if numarticles >= 1:
        ctk.CTkButton(self.top,
text=self.articles[startnum+1].getTitle(),
                        command=lambda:
self.displayURLcontent(self.articles[startnum+1])).place(x=50,
y=120)
        if numarticles >= 2:
            ctk.CTkButton(self.top,
text=self.articles[startnum+2].getTitle(),
                        command=lambda:
self.displayURLcontent(self.articles[startnum+2])).place(x=50,
y=170)
            if numarticles >= 3:
                ctk.CTkButton(self.top, text=self.articles[startnum
+ 3].getTitle(),
                        command=lambda:
self.displayURLcontent(self.articles[startnum + 3])).place(x=50,
y=220)
                if numarticles >= 4:
                    ctk.CTkButton(self.top,
text=self.articles[startnum + 4].getTitle(),
                        command=lambda:
self.displayURLcontent(self.articles[startnum + 4])).place(
x=50,
y=270)
                    if numarticles == 5:
                        ctk.CTkButton(self.top,
text=self.articles[startnum + 5].getTitle(),
                        command=lambda:
self.displayURLcontent(self.articles[startnum + 5])).place(
x=50,
y=320)

if self.page != self.pages-1:

```

```

        ctk.CTkButton(self.top, text="Next Page", command=lambda:
self.ShowNextPage()).place(x=350, y=450)
        if self.page != 0:
            ctk.CTkButton(self.top, text="Previous Page",
command=lambda: self.ShowPreviousPage()).place(x=150, y=450)
            if self.page == self.pages-1:
                ctk.CTkButton(self.top, text="Get More", command=lambda:
self.moreSearch()).place(x=350, y=450)

        self.searching = False
        self.search_button.configure(state="normal")

    def moreSearch(self):
        for widget in self.top.winfo_children():
            if widget.winfo_y() >= 80:
                widget.destroy()
        threading.Thread(target=self.searchUrls).start()

    def newSearch(self, query):
        print("searching")
        if self.searching:
            print("Already searching")
            return
        self.searching = True
        self.articles = []
        self.page = 0
        self.pages = 0
        self.query = query

        for widget in self.top.winfo_children():
            if widget.winfo_y() >= 80:
                widget.destroy()

        threading.Thread(target=self.searchUrls).start()

    def mainPage(self):
        self.urlsInRange = 0
        self.top = ctk.CTkToplevel(self.root)
        self.top.geometry("700x500")
        self.top.title("BarakVPN")
        self.top.iconbitmap("barak.ico")
        self.top.after(200, lambda: self.top.iconbitmap("barak.ico"))

        searchText = ctk.CTkEntry(self.top, placeholder_text="Enter
your news topic?", width=350)
        searchText.place(x=70, y=50)

        self.search_button = ctk.CTkButton(self.top, text="Return
results", command=lambda: self.newSearch(searchText.get()))
        self.search_button.place(x=440, y=50)

    def loginPage(self):
        l = LogIn(self.root, lambda: self.mainPage(), self.server)

```

```
l.login()

def run(self):
    if self.server.CheckIfOk():
        print("server connected")
        self.root.withdraw()
        self.loginPage()
    else:
        ctk.CTkLabel(self.root, text="UNABLE TO CONNECT TO SERVER",
font=("Arial", 30)).place(x=40, y=100)
        ctk.CTkLabel(self.root, text="please close the app and try
again later", font=("Arial", 20)).place(x=115, y=200)

        self.root.mainloop()

if __name__ == "__main__":
    app = CoolAslBarakVPN()
    app.run()
```

הקובץ האחראי על התחברות והרשמה למערכת - login.py

```
import customtkinter as ctk
from PIL import Image
import hashlib
```

```

class LogIn:
    def __init__(self, root, on_success, server):
        self.server = server
        self.root = ctk.CTkToplevel(root)
        self.root.title("BarakVPN")
        self.root.geometry('400x400')
        self.root.iconbitmap("barak.ico")
        self.root.after(200, lambda: self.root.iconbitmap("barak.ico"))

        self.error = None
        self.top = ctk.CTkToplevel(self.root)
        self.top.title("BarakVPN")
        self.top.iconbitmap("barak.ico")
        self.top.after(200, lambda: self.top.iconbitmap("barak.ico"))
        self.top.withdraw()
        self.on_success = on_success

    def toggle_password(self):
        if self.passwordE.cget("show") == "*":
            self.passwordE.configure(show="")
        else:
            self.passwordE.configure(show="*")

    def login(self):
        self.root.deiconify()
        self.top.withdraw()
        if self.error:
            self.error.destroy()
        ctk.CTkLabel(self.root, text="Welcome to BARAK VPN",
font=("Helvetica", 30)).place(x=30, y=70)

        ctk.CTkLabel(self.root, text="Username").place(x=100, y=150)
        ctk.CTkLabel(self.root, text="Password").place(x=100, y=200)
        self.username = ctk.CTkEntry(self.root, placeholder_text="enter
username here")
        self.username.place(x=200, y=150)
        self.passwordE = ctk.CTkEntry(self.root,
placeholder_text="enter password here", show="*")
        self.passwordE.place(x=200, y=200)
        ctk.CTkButton(self.root, text="show", fg_color="black",
text_color="white", hover_color="dark blue",
command=self.toggle_password,
width=30).place(x=350, y=200)

        login_button = ctk.CTkButton(self.root, text="LOG IN",
fg_color="black", text_color="white", hover_color="dark blue",
command=self.checkifexists)
        login_button.place(x=120, y=270)
        self.root.bind("<Return>", lambda event: login_button.invoke())

        ctk.CTkLabel(self.root, text="dont have an
account?").place(x=100, y=300)

```

```

        ctk.CTkButton(self.root, text="sign up here", fg_color='light
grey', hover_color='white', text_color="blue",
                        border_width=0, width=80,
command=self.signup).place(x=230, y=300)

    def DisplayInstructions(self):
        new = ctk.CTkToplevel(self.root)
        new.title("BarakVPN")
        new.iconbitmap("barak.ico")
        new.after(200, lambda: new.iconbitmap("barak.ico"))
        new.geometry("340x300")
        new.attributes("-topmost", True)
        new.protocol("WM_DELETE_WINDOW", lambda: new.destroy())
        ctk.CTkLabel(new, text="Password Instructions",
font=("Helvetica", 20)).place(x=70, y=20)
        ctk.CTkLabel(new, text="Passwords must:", font=("Helvetica",
15)).place(x=110, y=50)
        ctk.CTkLabel(new, text="contain 8 or more letters").place(x=30,
y=100)
        ctk.CTkLabel(new, text="contain at least one upper case
letter").place(x=30, y=120)
        ctk.CTkLabel(new, text="contain at least one lower case
letter").place(x=30, y=140)
        ctk.CTkLabel(new, text="contain at least one
number").place(x=30, y=160)
        ctk.CTkLabel(new, text="cant have a series of at least 3
following numbers").place(x=30, y=180)
        ctk.CTkLabel(new, text="contain the same character 3 or more
times in a row").place(x=30, y=200)

    def signup(self):
        if self.error:
            self.error.destroy()
        self.root.withdraw()
        self.top.deiconify()
        self.top.geometry('400x400')
        ctk.CTkLabel(self.top, text="Welcome to BARAK VPN",
font=("Helvetica", 30)).place(x=30, y=50)
        ctk.CTkLabel(self.top, text="sign up for the best project in
the world here").place(x=100, y=80)

        ctk.CTkLabel(self.top, text="enter your email").place(x=50,
y=150)
        ctk.CTkLabel(self.top, text="enter your username").place(x=50,
y=200)
        ctk.CTkLabel(self.top, text="enter your password").place(x=50,
y=250)

        self.email = ctk.CTkEntry(self.top, placeholder_text="enter
email here")
        self.email.place(x=200, y=150)
        self.username = ctk.CTkEntry(self.top, placeholder_text="enter
username here")

```

```

        self.username.place(x=200, y=200)
        self.passwordE = ctk.CTkEntry(self.top, placeholder_text="enter
password here", show="*")
        self.passwordE.place(x=200, y=250)

        ctk.CTkButton(self.top, text="show", fg_color="black",
text_color="white", hover_color="dark blue",
                        command=self.toggle_password,
width=30).place(x=350, y=250)

        image = Image.open("question_mark.png")
        image = ctk.CTkImage(light_image=image, dark_image=image,
size=(10, 10))

        ctk.CTkButton(self.top, text="", image=image, fg_color="white",
hover_color="blue",
                        command=self.DisplayInstructions,
width=30).place(x=10, y=250)

        signup_button = ctk.CTkButton(self.top, text="SIGN UP",
fg_color="black", text_color="white", hover_color="dark blue",
                        command=self.checkifcancreate)
        signup_button.place(x=120, y=320)
        self.top.bind("<Return>", lambda event: signup_button.invoke())

        ctk.CTkLabel(self.top, text="already have an
account?").place(x=90, y=350)
        ctk.CTkButton(self.top, text="log in here", fg_color='light
grey', hover_color='white', text_color="blue",
                        border_width=0, width=80,
command=self.login).place(x=240, y=350)

    def check_password(self, password):
        if len(password) <= 8:
            return False
        if not any(c.isupper() for c in password):
            return False
        if not any(c.islower() for c in password):
            return False
        if not any(c.isdigit() for c in password):
            return False
        for i in range(len(password) - 3):
            if password[i] == password[i + 1] == password[i + 2] ==
password[i + 3]:
                return False

        for i in range(len(password) - 2):
            if password[i].isdigit() and password[i + 1].isdigit() and
password[i + 2].isdigit():
                if int(password[i + 1]) == int(password[i]) + 1 and
int(password[i + 2]) == int(password[i + 1]) + 1:
                    return False

```

```

        return True

    def checkifcancreate(self):
        if self.error:
            self.error.destroy()

        password = self.passwordE.get()
        username = self.username.get()
        mail = self.email.get()

        answer = "weak password"

        if not username or not password or not mail:
            answer = "cannot create"

        elif self.check_password(password):
            hash_object = hashlib.sha256(password.encode())
            hash_hex = hash_object.hexdigest()

            tosend = f"signup,{username},{hash_hex},{mail}"
            answer = self.server.SendData(tosend)

        if answer == "True":
            self.top.withdraw()
            self.on_success()
        else:
            if answer == "email problem":
                self.error = ctk.CTkLabel(self.top, text="invalid email address", font=("Helvetica", 10),
                                           text_color="red")
                self.error.place(x=135, y=285)
            elif answer == "forbidden words":
                self.error = ctk.CTkLabel(self.top, text="please use prettier words😏", font=("Helvetica", 10), text_color="red")
                self.error.place(x=135, y=285)
            elif answer == "weak password":
                self.error = ctk.CTkLabel(self.top, text="please follow the password manual", font=("Helvetica", 10),
                                           text_color="red")
                self.error.place(x=115, y=285)
            elif answer == "cannot create":
                self.error = ctk.CTkLabel(self.top, text="please fill all the parameters", font=("Helvetica", 10),
                                           text_color="red")
                self.error.place(x=130, y=285)
            elif answer == "error":
                self.top.withdraw()

        top = ctk.CTkToplevel(self.top)
        top.geometry("600x400")
        top.title("BarakVPN")
        top.iconbitmap("barak.ico")
        top.after(200, lambda: top.iconbitmap("barak.ico"))

```

```

        ctk.CTkLabel(top, text="UNABLE TO CONNECT TO SERVER",
font=("Arial", 30)).place(x=40, y=100)
        ctk.CTkLabel(top, text="please close the app and try
again later", font=("Arial", 20)).place(x=115,
y=200)

        top.mainloop()

    else:
        self.error = ctk.CTkLabel(self.top, text="User already
exists", font=("Helvetica", 10),
                                text_color="red")
        self.error.place(x=135, y=285)

    def VerifyMail(self, mail):
        self.verify = ctk.CTkToplevel(self.root)
        self.verify.geometry("300x250")
        self.verify.title("BarakVPN")
        self.verify.iconbitmap("barak.ico")
        self.verify.after(200, lambda:
self.verify.iconbitmap("barak.ico"))

        self.verify.attributes("-topmost", True)
        ctk.CTkLabel(self.verify, text=f"enter the code you have been
sent to \n {mail}", font=("Helvetica", 15)).place(x=20, y=20)
        ctk.CTkLabel(self.verify, text="make sure to check the spam
folder too").place(x=40, y=60)
        self.codeEntry = ctk.CTkEntry(self.verify,
placeholder_text="enter your code here")
        self.codeEntry.place(x=80, y=90)

        ctk.CTkButton(self.verify, text="Verify", width=100,
fg_color="black", hover_color="dark blue",
command=self.Check).place(x=90, y=160)

    def Check(self):
        if self.code == self.codeEntry.get():
            print("log in successful!")
            self.verify.withdraw()
            self.on_success()
        else:
            ctk.CTkLabel(self.verify, text="incorrect code",
font=("Helvetica", 10), text_color="red").place(x=110,
y=125)

    def RSAdecrypt(self, pk, ciphertext):
        key, n = pk

        ciphertext = ciphertext.strip("[]")

```



```

        ciphertext = [int(letter.strip()) for letter in
ciphertext.split(",")]

        plain = [chr((char ** key) % n) for char in ciphertext]
        return ''.join(plain)

    def checkifexists(self):
        password = self.passwordE.get()
        username = self.username.get()

        if not username or not password:
            answer = "False"

        else:
            hash_object = hashlib.sha256(password.encode())
            hash_hex = hash_object.hexdigest()

            tosend = f"login,{username},{hash_hex}"
            answer = self.server.SendData(tosend)

            if "True" in answer:
                self.root.withdraw()
                ans = answer.split('/')
                self.code = self.RSAdecrypt((5213,7373), str(ans[1]))
                mail = self.RSAdecrypt((5213,7373), ans[2])
                self.VerifyMail(mail)

            elif answer == "error":
                self.root.withdraw()

                top = ctk.CTkToplevel(self.root)
                top.geometry("600x400")
                top.title("BarakVPN")
                top.iconbitmap("barak.ico")
                top.after(200, lambda: top.iconbitmap("barak.ico"))

                ctk.CTkLabel(top, text="UNABLE TO CONNECT TO SERVER",
font=("Arial", 30)).place(x=40, y=100)
                ctk.CTkLabel(top, text="please close the app and try again
later", font=("Arial", 20)).place(x=115,
y=200)
                top.mainloop()

            else:
                self.error = ctk.CTkLabel(self.root, text="Account doesn't
exist", font=("Helvetica", 10), text_color="red")
                self.error.place(x=140, y=230)

```

קובץ אחראי על תקשורת עם השרת - communication.py

```
import pickle
import socket
import multiprocessing
from article import Article
from flask import Flask, Response, request
import queryType
import random
import ssl
```

```

class Communication():
    def __init__(self, serverIp, port):
        self.host = serverIp
        self.port = port
        self.isOk = True
        self.flask_process = None
        self.client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        try:
            self.client.settimeout(0.1)
            self.client.connect((self.host, self.port))
            msg = self.client.recv(1024).decode()
            if msg == "server_full":
                print("Server is full. Connection closed.")
                self.client.close()
                self.isOk = False

            else:
                self.client.settimeout(None)
                print("Successfully connected to server.")

        except Exception as e:
            print(f"Failed to connect: {e}")
            self.client.close()
            self.isOk = False

    def CheckIfOk(self):
        return self.isOk

    def RSAencrypt(self, pk, plaintext):
        key, n = pk
        cipher = [(ord(char) ** key) % n for char in plaintext]
        return cipher

    def decrypt(self, text, code1, code2):
        decrypted = ""
        for i in range(2):
            if i == 0:
                code = code1
                half = text[:len(text) // 2]
            else:
                code = code2
                half = text[len(text) // 2:]
            result = ''
            for char in half:
                if 'a' <= char <= 'z':
                    result += chr((ord(char) - ord('a') - code) % 26 +
ord('a'))
                elif 'A' <= char <= 'Z':

```

```

        result += chr((ord(char) - ord('A') - code) % 26 +
ord('A'))
    else:
        result += char
    decrypted += result
    return decrypted

def GetUrls(self, query, start, titles):
    if query == "":
        return []

    if "https://" in query:
        formater = queryType.UrlQuery(query, start)
    else:
        formater = queryType.RegularQuery(query, start)

    formater.setQuery(self.RSAencrypt((1877, 7373), query))

    query = formater.format_for_sending()
    try:
        self.client.send(query)

        for i in range(len(titles)):
            titles[i] = self.RSAencrypt((1877, 7373), titles[i])

        data = pickle.dumps(titles)
        data_length = len(data)
        self.client.sendall(data_length.to_bytes(8, 'big'))
        self.client.sendall(data)

        data_length_bytes = self.client.recv(8)
        data_length = int.from_bytes(data_length_bytes, 'big')

        received_data = b""

        while len(received_data) < data_length:
            msg = self.client.recv(4096)
            if not msg:
                break
            received_data += msg

        articles = pickle.loads(received_data)
    except Exception as e:
        print(f"Error getting urls: {e}")
        return []

    print("Finished receiving all articles.")

    return articles

def SendData(self, message):
    try:

```

```

        if "signup" in message:
            toEncrypt = message.split(",")
            message = f"siGnUp/{self.RSAencrypt((1877, 7373),
toEncrypt[1])}/{toEncrypt[2]}/{self.RSAencrypt((1877, 7373),
toEncrypt[3])}"
            elif "login" in message:
                toEncrypt = message.split(",")
                message = f"loGIn/{self.RSAencrypt((1877, 7373),
toEncrypt[1])}/{toEncrypt[2]}"
                self.client.send(message.encode())

        response = self.client.recv(1024).decode()

    except Exception as e:
        return "error"
    return response

def gather_content(self, base_url, path):
    rec, msg = "", ""

    full_url = f"{base_url}/{path}"
    if request.query_string:
        print(request.query_string.decode('utf-8'))
        full_url += '?' + request.query_string.decode('utf-8')

    code1 = random.randint(1,25)
    code2 = random.randint(1,25)

    self.client.sendall(f"oPen~{self.RSAencrypt((1877,7373),
full_url)}~{self.RSAencrypt((1877,7373),
str(code1))}~{self.RSAencrypt((1877,7373), str(code2))}".encode())

    while "finished_sending_content" not in msg:
        rec = self.client.recv(4096).decode(errors="ignore")
        if "failed" in rec:
            if msg == "failed".encode():
                return (None, None, None)
        msg += rec

    parts = msg.split("revah")
    content = parts[0]
    content = self.decrypt(content, code1, code2)
    status = int(parts[1])
    content_type = parts[2]

    return content, status, content_type

def create_app(self, url):
    app = Flask(__name__)

    @app.route('/', defaults={'path': ''})
    @app.route('/<path:path>')
    def show_content(path):

```

```

        try:
            content, status_code, content_type =
self.gather_content(url, path)
            if not content and not status_code and not
content_type:
                raise Exception("couldn't fetch this part")
            return Response(content, status=status_code,
content_type=content_type)
        except Exception as e:
            return Response(str(e), status=502,
content_type='text/plain')

    return app

def run_flask(self, url):
    app = self.create_app(url)
    app.run(port=5000, debug=False, use_reloader=False)

def OpenInBrowser(self, url):
    if self.flask_process and self.flask_process.is_alive():
        self.flask_process.terminate()
        self.flask_process.join()

    print(f"Starting new Flask app for {url}")
    self.flask_process =
multiprocessing.Process(target=self.run_flask, args=(url,))
    self.flask_process.start()
    return self.flask_process

```

קובץ אחראי על יצירת בקשות חיפוש לשרת - queryType.py

```

class QueryType:
    def __init__(self, query, start):
        self.query = query
        self.start = start

    def setQuery(self, query):
        self.query = query

class RegularQuery(QueryType):
    def __init__(self, query, start):

```

```
super().__init__(query, start)

def format_for_sending(self):
    return f"uRl/search/{self.query}/{self.start}".encode()

class UrlQuery(QueryType):
    def __init__(self, query, start):
        super().__init__(query, start)

    def format_for_sending(self):
        return f"uRl/url/{self.query}/{self.start}".encode()
```

קובץ אחראי על שמירת הכתבות בקבצי טקסט במחשב - save.py

```
import os

class Save():
    def __init__(self, title, content, url):
        self.title = title
        self.content = content
        self.url = url

    def cleanTitle(self):
```

```
invalid_chars = ['\\', '/', ':', '*', '?', '"', '<', '>', '|']
for char in invalid_chars:
    self.title = self.title.replace(char, '_')
return self.title

def saveToComputer(self):
    try:
        self.title = self.cleanTitle()
        with open(f"{self.title}.txt", mode="w",
encoding="utf-8-sig") as file:
            file.write(self.url)
            file.write("\n\n\n\n")
            file.write(self.content)

        os.startfile(f"{self.title}.txt")
        return True
    except:
        return False
```

קובץ אחראי לטיפול בכתבות - article.py

```
import requests
from bs4 import BeautifulSoup
import random

class Article:
    def __init__(self, url, query):
        self.url = url
        self.query = query
        self.content = ""
```



```

        self.status_code = 0
        self.title = ""
        self.code = random.randint(1,25)
        self.code2 = random.randint(1,25)

    def getUrl(self):
        return self.url

    def getContent(self):
        return self.content

    def getTitle(self):
        return self.title

    def getCode(self):
        return self.code

    def getCode2(self):
        return self.code2

    def setUrl(self, t):
        self.url = t

    def setContent(self, t):
        self.content = t

    def setTitle(self, t):
        self.title = t

    def setCode(self, t):
        self.code = t

    def setCode2(self, t):
        self.code2 = t

    def RSAencrypt(self, pk, plaintext):
        key, n = pk
        cipher = [(ord(char) ** key) % n for char in plaintext]
        return cipher

    def GetInfo(self):
        headers = {
            "User-Agent": (
                "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                "AppleWebKit/537.36 (KHTML, like Gecko) "
                "Chrome/115.0.0.0 Safari/537.36"
            )
        }

        try:
            response = requests.get(self.url, headers=headers,
timeout=10)
        except requests.RequestException as e:

```

```

        print(f"error in article: Request failed for {self.url}:
{e}")

        return False

    soup = BeautifulSoup(response.content, 'html.parser')

    title_tag = soup.find("title")
    self.title = ""
    if title_tag:
        self.title = title_tag.get_text(strip=True)

    content_parts = []

    for heading in soup.find_all(['h1', 'h2', 'h3', 'h4', 'h5',
'h6']):
        content_parts.append(heading.get_text(strip=True)+"\n")

    for paragraph in soup.find_all('p'):
        content_parts.append(paragraph.get_text(strip=True)+"\n")

    self.content = ' '.join(content_parts).strip()

    if content_parts == []:
        self.content = "this article doesn't contain any text
information, try to open it in browser"

    if response.status_code != 200 or not self.title:
        return False

    blocked_titles = [
        "error", "403", "no title found for this article",
"homepage",
        "just a moment", "captcha", "blocked"
    ]

    blocked_content_words = [
        "access denied", "blocked", "please verify", "security
check",
        "captcha", "robot", "user-agent"
    ]

    title_lower = self.title.lower()
    content_lower = self.content.lower()

    for word in blocked_titles:
        if word in title_lower and word not in self.query.lower():
            return False

    for word in blocked_content_words:
        if word in content_lower:
            return False
    return True

```

קובץ השרת - server.py

```
import multiprocessing
import pickle
import socket
import threading
from googlesearch import search
from duckduckgo_search import DDGS
import select
from article import Article
import requests
```

```

from createDataBase import DataBase
import smtplib
import random
import time

def encrypt(text, code1, code2):
    encrypted = ""
    for i in range(2):
        if i == 0:
            code = code1
            half = text[:len(text) // 2]
        else:
            code = code2
            half = text[len(text) // 2:]

        result = ''
        for char in half:
            if 'a' <= char <= 'z':
                result += chr((ord(char) - ord('a') + code) % 26 +
ord('a'))
            elif 'A' <= char <= 'Z':
                result += chr((ord(char) - ord('A') + code) % 26 +
ord('A'))
            else:
                result += char
        encrypted += result

    return encrypted

def RSAdecrypt(pk, ciphertext):
    key, n = pk
    ciphertext = ciphertext.strip("[]")
    ciphertext = [int(letter.strip()) for letter in
ciphertext.split(",")]

    plain = [chr((char ** key) % n) for char in ciphertext]

    decrypted = ''.join(plain)

    return decrypted

def check_if_okay(text):
    counterparts = {
        '4': 'a', '3': 'e', '1': 'i', '0': 'o', '5': 's', '7': 't',
'8': 'b',
        '9': 'g', '@': 'a', '$': 's', '%': 's', '&': 'e', '+': 't',
        '!': 'l', '#': 'h', '*': 'x', '^': 'v',
        '(': 'c', ')': 'c', '[': 'e', ']': 'e',
        ':': 'i', ';': 'i', '_': ' ', '-': ' ', '|': 'l',
        '\': 'a', '"': 'i', '<': 'c', '>': 'c', '.': ' '
    }
    banned_words = [

```

```

        "fuck", "shit", "bitch", "asshole", "dick", "cunt", "nigger",
        "faggot", "retard",
        "whore", "slut", "rape", "rapist", "motherfucker", "bastard",
        "crap", "bbc",
        "piss", "cock", "pussy", "twat", "cum", "jizz", "fag", "hoe",
        "kill", "murder", "terrorist",
        "drug", "weed", "cocaine", "heroin", "meth", "porn", "sex",
        "xxx", "boobs", "pussy", "tits",
        "milf", "dildo", "blowjob", "handjob", "hitler", "adolf",
        "nazi", "kkk", "whitepower", "islamist",
        "racist", "bigot", "homo", "twink", "ballerina capucina",
        "science based lifer", "shai skladman", "homophobe",
        "misogynist", "kys", "suicide", "selfharm",
        "zoophile", "necrophilia", "bestiality", "incest"
    ]

    text = text.lower()

    for change, normal in counterparts.items():
        text = text.replace(change, normal)

    for word in banned_words:
        if word in text:
            return False

    return True

def checkMail(mail):
    valid_domains = [
        "gmail.com", "yahoo.com", "outlook.com", "hotmail.com",
        "walla.co.il", "bezeqint.net",
        "aol.com", "icloud.com", "protonmail.com", "zoho.com",
        "mail.com",
        "gmx.com", "yandex.com", "me.com", "msn.com", "live.com",
        "comcast.net", "att.net",
        "verizon.net", "ymail.com", "qq.com", "naver.com",
        "hotmail.co.uk",
        "outlook.co.uk", "fastmail.com", "pm.me",
    ]

    if "@" not in mail:
        return False

    local_part, domain_part = mail.split("@")

    if not local_part or not domain_part:
        return False

    if "." not in domain_part:
        return False

    if any(c.isupper() for c in mail):

```

```

        return False

    if domain_part not in valid_domains:
        return False

    allowed_chars =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789._-"
    for char in local_part + domain_part:
        if char not in allowed_chars:
            return False

    return True

def checkIfCanAdd(client_socket, data):
    data = data.split("/")
    username = RSADecrypt((5213, 7373), data[1])
    password = data[2]
    email = RSADecrypt((5213, 7373), data[3])

    dataBase = DataBase()

    if not check_if_okay(username):
        response = "forbidden words"

    elif not checkMail(email):
        response = "email problem"

    elif not dataBase.CheckIfCanAdd(username, email):
        response = "False"

    else:
        dataBase.AddUser(username, password, email)

        print("SignUp successful!")
        response = "True"

    client_socket.send(response.encode())

def CheckifExists(client_socket, data):
    data = data.split("/")
    username = RSADecrypt((5213, 7373), data[1])
    password = data[2]

    dataBase = DataBase()
    user_data = dataBase.CheckIfExists(username, password)

    if user_data:
        mail = dataBase.GetEmailByUsername(username)
        code = sendVerifactionMail(mail)

        if code != 0:

```

```

        print("Login successful!")

        client_socket.send(f"True/{RSAencrypt((1877, 7373),
str(code))}/{RSAencrypt((1877, 7373), mail)}".encode())
    else:
        client_socket.send("False".encode())
else:
    client_socket.send("False".encode())

def sendVerifactionMail(mail):
    code = random.randint(100000, 999999)
    sender_email = "barakvpnoofficial@gmail.com"
    receiver_email = mail
    password = "rrpsovtfbwbznmuo"
    subject = "Verification Code"
    body = f"Your verification code for BarakVPN is: {code}"

    message = f"""\
From: {sender_email}
To: {receiver_email}
Subject: {subject}

{body}
"""

    try:
        with smtplib.SMTP("smtp.gmail.com", 587) as server:
            server.starttls()
            server.login(sender_email, password)
            server.sendmail(sender_email, receiver_email, message)
    except Exception as e:
        print("Failed to send email:", e)
        return 0

    return code

def gather_content(client_socket, data):
    if data.count("oPen") > 1:
        raw_commands = data.split("oPen")
        gather_content(client_socket, "oPen"+raw_commands[1])
        for i in range(2, len(raw_commands)):
            if raw_commands[i] != "":
                gather_content(client_socket, "oPen" + raw_commands[i])
    else:
        try:
            data = data.split("~")
            full_url = RSAdecrypt((5213,7373), data[1])
            code1 = int(RSAdecrypt((5213,7373), data[2]))
            code2 = int(RSAdecrypt((5213,7373), data[3]))

```

```

        response = requests.get(full_url)
        content = response.content.decode(errors="ignore")
        content = encrypt(content, code1, code2)
        status = str(response.status_code)
        print(full_url + ": " + status)
        content_type = response.headers.get("Content-Type",
"text/html")

        msg =
f"{content}revah{status}revah{content_type}revahfinished_sending_conte
nt"

        client_socket.sendall(msg.encode())
    except Exception as e:
        client_socket.sendall("failed".encode())

def HandleClient(client_socket, client_address, remove):
    while True:
        try:
            data = client_socket.recv(8192).decode()
            if data == "":
                continue
            print(data)
            if "siGnUp" in data:
                checkIfCanAdd(client_socket, data)
            elif "loGIn" in data:
                CheckifExists(client_socket, data)
            elif "uRl" in data:
                ReturnInfo(client_socket, data)
            elif "oPen" in data:
                gather_content(client_socket, data)
            else:
                pass
        except ConnectionResetError as e:
            print("lost connection from ", client_address, " ", e)
            remove()
            break

def RemoveClient(client):
    print("Removing", client)
    if client in clients:
        clients.remove(client)
        client.close()

def RSAencrypt(pk, plaintext):
    key, n = pk
    cipher = [(ord(char) ** key) % n for char in plaintext]
    return cipher

```



```

def SingleArticle(query, url, articles, titles):
    a = Article(url, query)
    if a.GetInfo():
        if a.getTitle() not in titles:
            a.setTitle(RSAencrypt((1877, 7373), a.getTitle()))
            a.setUrl(RSAencrypt((1877, 7373), a.getUrl()))

            a.setContent(encrypt(a.getContent(), a.getCode(),
a.getCode2()))

            a.setCode(a.RSAencrypt((1877, 7373), str(a.getCode())))
            a.setCode2(a.RSAencrypt((1877, 7373), str(a.getCode2())))
            articles.append(a)

def ReturnInfo(client_socket, data):
    data = data.split('/')
    query = RSAdecrypt((5213, 7373), data[2])

    data_length = int.from_bytes(client_socket.recv(8), 'big')
    received_data = b""
    while len(received_data) < data_length:
        chunk = client_socket.recv(min(4096, data_length -
len(received_data)))
        received_data += chunk

    titles = pickle.loads(received_data)

    if not check_if_okay(query):
        articles = []
        articlesInBytes = pickle.dumps(articles)
        data_length = len(articlesInBytes)
        client_socket.sendall(data_length.to_bytes(8, 'big'))

        client_socket.sendall(articlesInBytes)
        print("unethical")
        return

    for i in range(len(titles)):
        plain = [chr((char ** 5213) % 7373) for char in titles[i]]
        titles[i] = ''.join(plain)

    if data[1] == "search":
        start = int(data[3])
        if start >= 30:
            articles = []

            articlesInBytes = pickle.dumps(articles)
            data_length = len(articlesInBytes)
            client_socket.sendall(data_length.to_bytes(8, 'big'))

            client_socket.sendall(articlesInBytes)

```

```

        return

    processes = []
    articles = multiprocessing.Manager().list()
    time.sleep(random.uniform(0.5, 2.5))
    try:
        urls = list(search("news about:" + query, start_num=start,
num_results=15, region="us"))
        if "" in urls:
            raise Exception("google giving empty links")
    except Exception as e:
        print("google doesn't work", e)

    try:
        ddgs = DDGS()
        results = ddgs.text("news about:" + query,
max_results=15+start, region = "us")
        urls = [result['href'] for result in results]

    except Exception as e:
        print("duckduckgo doesnt work", e)
        urls = []

    i = -1
    for url in urls:
        i+=1
        p = multiprocessing.Process(target=SingleArticle,
args=(query, url, articles, titles))
        p.start()
        processes.append(p)

    for p in processes:
        p.join()

    articlesInBytes = pickle.dumps(list(articles))

else:
    articles = []
    SingleArticle(query, query, articles, titles)

    articlesInBytes = pickle.dumps(articles)

data_length = len(articlesInBytes)
client_socket.sendall(data_length.to_bytes(8, 'big'))

client_socket.sendall(articlesInBytes)
print("Finished sending articles", len(articles))

def main():
    host = '127.0.0.1'

```

```

port = 5555

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((host, port))
server.listen()

global clients
clients = []
threads = []

print(f"Server listening on {host}:{port}")

while True:
    rlist, wlist, xlist = select.select([server] + clients,
clients, [])

    for client in rlist:
        if client is server:

            client_socket, client_address = client.accept()
            if len(clients) >= 10:

                print(f"Rejected connection from {client_address} -
server full.")

                client_socket.send("server_full".encode())
                client_socket.close()
            else:
                print(f"New connection from {client_address}")
                client_socket.send("connected".encode())
                clients.append(client_socket)

                client_thread = threading.Thread(
                    target=HandleClient,
                    args=(client_socket, client_address, lambda
socket=client_socket: RemoveClient(socket))
                )
                client_thread.start()
                threads.append(client_thread)

if __name__ == "__main__":
    main()

```

[createDataBase.py](#) - קובץ אחראי על עבודה עם מסד הנתונים

```
import sqlite3

class DataBase:
    def __init__(self, db_path='credentials.db'):
        self.db_path = db_path
        self.conn = sqlite3.connect(self.db_path)
        self.cursor = self.conn.cursor()
        self.create_table()
```

```

def create_table(self):
    self.cursor.execute('''
        CREATE TABLE IF NOT EXISTS logins (
            username TEXT NOT NULL UNIQUE,
            password TEXT NOT NULL,
            email TEXT NOT NULL UNIQUE
        )
    ''')
    self.conn.commit()

def CheckIfCanAdd(self, username, mail):
    self.cursor.execute("SELECT * FROM logins WHERE username=?",
(username,))
    if self.cursor.fetchone() is None:
        print("username not found")
        self.cursor.execute("SELECT * FROM logins WHERE email=?",
(mail,))
        if self.cursor.fetchone() is None:
            print("email not found")
            return True
        return False
    return False

def AddUser(self, username, password, email):
    try:
        self.cursor.execute("INSERT INTO logins (username,
password, email) VALUES (?, ?, ?)", (username, password, email))
        self.conn.commit()
        return True
    except:
        return False

def CheckIfExists(self, username, password):
    self.cursor.execute("SELECT * FROM logins WHERE username=? AND
password=?", (username, password))
    if self.cursor.fetchone() is None:
        return False
    return True

def GetEmailByUsername(self, username):
    self.cursor.execute("SELECT email FROM logins WHERE
username=?", (username,))
    result = self.cursor.fetchone()
    if result:
        return result[0]
    return None

```

